

MOUAZ ALJOMAAT

الرحلة نحو إتقان FastAPI



رحلة اتقان FastAPI

دليل عملي للأداء العالي API & Microservices

النسخة الأولى

معاذ الجمعيات

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



المحتويات

6 لماذا FastAPI ؟

❖ مقدمة ل FastAPI واهميتها..... 6

❖ مقارنتها مع الأطر الأخرى (Django, Flask)..... 8

14 ابدأ أول تطبيق لك باستخدام FastAPI

❖ متطلبات النظام وأفضل الممارسات..... 14

❖ تثبيت FastAPI و شغل أول تطبيق لك..... 15

❖ تشغيل أول واجهة برمجية (API) والوصول إليها عبر المتصفح (Browser)..... 18

❖ استخدام Swagger UI و ReDoc لتوثيق ال API..... 20

❖ أساسيات بناء ال APIs..... 22

❖ المعاملات الديناميكية في المسار (Dynamic Path Parameters)..... 27

❖ معاملات المسار (Path Parameters) و معاملات الاستعلام..... 28

❖ (Query Parameters)..... 28

33 التحقق من صحة البيانات باستخدام Pydantic

❖ بناء نماذج البيانات في FastAPI..... 33

❖ استخدام Pydantic لإنشاء النماذج..... 33

❖ التحقق من البيانات ومعالجة الأخطاء..... 35

❖ الحقول الافتراضية (Default Fields) والحقول الإلزامية (Required Fields)..... 38

❖ التعامل مع الطلبات والاستجابات..... 42

❖ تخصيص استجابات HTTP (Customizing HTTP Responses)..... 44

❖ استخدام HTTPException لمعالجة الأخطاء (Using HTTPException for Error).....

52 تحسين الأداء باستخدام البرمجة غير المتزامنة

52.....	فهم آلية Async/Await في FastAPI
59	العمل مع قواعد البيانات
59.....	الاتصال بقاعدة البيانات باستخدام SQLAlchemy
71.....	التعامل مع الأخطاء أثناء عمليات قاعدة البيانات
77	أفضل الممارسات لتصميم واجهات البرمجة
77.....	هيكلية مشروع (FastAPI) قابل للتوسع
86.....	الأمان في (FastAPI)
95	بناء الخدمات المصغرة باستخدام FastAPI
95.....	تصميم الخدمات المصغرة باستخدام FastAPI
104.....	التواصل بين الخدمات (Inter-Service Communication):
112	المراجعة النهائية ومصادر التعلم الإضافية
112.....	ملخص النقاط الرئيسية
114.....	مصادر إضافية لإتقان
115.....	استكشاف المواضيع المتقدمة
117	المراجع

المقدمة

في عالم البرمجة الحديث، تزداد الحاجة إلى تطبيقات عالية الأداء ومرنة قادرة على التعامل مع آلاف أو حتى ملايين الطلبات في الثانية الواحدة. وهنا تبرز أهمية الأدوات والأطر التي تمكن المطورين من بناء واجهات برمجة التطبيقات (APIs) بسرعة وكفاءة، مع الحفاظ على الأداء العالي (High Performance) وقابلية التوسع (Scalability) ومن بين هذه الأدوات، يبرز FastAPI كأحد أبرز الأطر المثالية لبناء واجهات API باستخدام لغة Python.

حول هذا الكتاب

يهدف هذا الكتاب إلى تقديم دليل سريع وعملي لمطوري Python لفهم كيفية استخدام FastAPI في تصميم واجهات برمجة التطبيقات (APIs)، وكيفية الاستفادة من ميزة async/await لتحسين الأداء ومعالجة عدد كبير من الطلبات في الوقت نفسه.

سيغطي هذا الدليل كل ما تحتاج إليه من:

- ◀ عملية التثبيت والإعداد (Installation & Setup)
- ◀ أساسيات بناء ال APIs
- ◀ بناء النماذج (Models) والتعامل مع البيانات (Data Handling)
- ◀ تحسين الأداء باستخدام Async
- ◀ التعرف على الخدمات المصغرة (Microservices) وكيفية التعامل معها
- ◀ تنظيم هيكل المشروع بفعالية (Project Structuring)

يُمزج الكتاب بين الشرح النظري والأمثلة العملية، ليمنحك فهماً تطبيقياً سريعاً وواضحاً لـ FastAPI، مما يجعله خياراً مثالياً للمطورين الذين يرغبون في بناء واجهات برمجة عالية الأداء وفعّالة خلال وقتٍ قصير.

سواء كنت مبتدئاً في تطوير الواجهات أو مطوراً متمرساً، فإن هذا الكتاب سيزودك بالأدوات والمعرفة العملية التي تحتاجها بإذن الله.

للملاحظات حول الكتاب:

البريد الإلكتروني: moazzjom@gmail.com

ال LinkedIn: [اضغط هنا](#)

الوحدة 1

مقدمة FastAPI

ملذا FastAPI ؟

❖ مقدمة ل FastAPI وأهميتها

FastAPI هي إطار حديث عالي الأداء لبناء واجهة برمجة التطبيقات (APIs) عن طريق بايثون (Python)، مصمم ليكون سريع، سهل الاستخدام، وفعال، FastAPI مبني على [starlette](#) لمعالجة الويب (Web Handling) واستخدام مكتبة Pydantic للتحقق من صحة البيانات (Data Validation)، مما يجعلها خيارًا قويًا لتطوير واجهات RESTful APIs و نمط معماري الخدمات المصغرة (Microservices).

وتكمن قوة ال FastAPI في أنها تدعم البرمجة الغير متزامنة (Asynchronous Programming) بشكل مدمج وأساسي، مما يتيح للمطورين بناء تطبيقات قابلة للتوسع (Scalable Applications) وقادرة على معالجة عدة طلبات بشكل غير متزامن (Asynchronously) مما يجعل التطبيق عالي الكفاءة والأداء، ويُعد ذلك مفيدًا بشكل خاص للتطبيقات التي تتطلب شبكات عالية الأداء (High-Performance Network-ing)، مثل الأنظمة الآتية (Real-Time Systems)، وتطبيقات إنترنت الأشياء (IoT Applications)، والخدمات المعتمدة على الذكاء الاصطناعي (AI-Driven Services).

أهم ميزات FastAPI :

أداء عالي: تقارن بإطار Node.js و Go في السرعة، وهذا كله بفضل قدرته في العمليات الغير متزامنة (Asynchronous).

التحقق التلقائي من البيانات : تعتمد على مكتبة [Pydantic](#) للتحقق من الطلبات (Request Validation) وإيضاً ال Serialization و Deserialization وهي عملية تحويل البيانات (Data) من شكلها الأصلي (مثلاً كائن/كلاس في Python) إلى صيغة يمكن إرسالها أو تخزينها بسهولة مثلاً صيغة JSON وهي الأشهر والعكس.

التوثيق المدمج (Built-in Documentation) : توفر بشكل تلقائي OpenAPI، (Swagger UI) و (ReDoc).

سهل الاستخدام والقراءة (Easy to Use and Read) : تتميز ببنية بسيطة، ويستفيد من ميزة تلميحات الأنواع في Python (Type Hints) لزيادة وضوح الكود وتحسين قراءته.

دعم العمليات المتزامنة وغير المتزامنة (Asynchronous and Synchronous): يتعامل بكفاءة مع كلٍ من العمليات المتزامنة (Sync) وغير المتزامنة (Async).

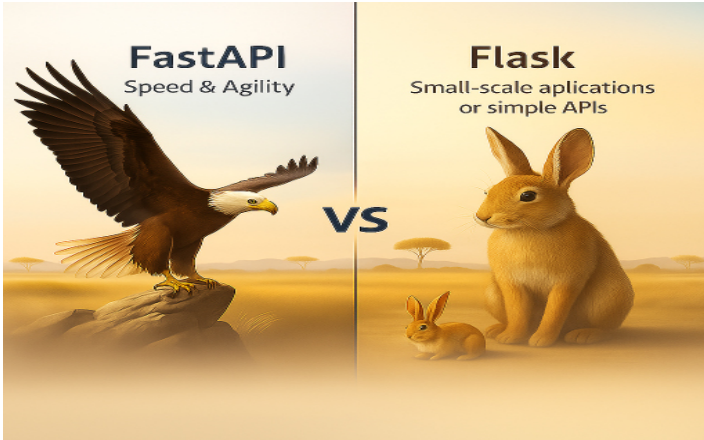
نظام حقن التبعيات (Dependency Injection System): يوفر أسلوبًا منظمًا لإدارة التبعيات داخل التطبيق، مما يجعل الكود أكثر وضوحًا وقابل للاختبار (Testable)، فهو يسمح بمشاركة الخدمات (مثل قواعد البيانات أو المصادقة) بين أجزاء مختلفة من التطبيق دون الحاجة إلى إعادة إنشائها في كل مرة، الأمر الذي يجعل الصيانة والتوسع أسهل بكثير.

جاهز للإنتاج (Production-Ready): صُمم ليتناسب مع بيئات الخدمات المصغرة (Microservices) الحديثة والهيكل المعتمدة على السحابة (Cloud-Based Architectures).

FastAPI هو خيار رائع للمطورين الذين يريدون بناء واجهات برمجية (APIs) صلبة وقوية وقابلة للتوسع وبالإضافة إلى ذلك بمجهود قليل فهو يقدم مزيجًا متينًا من السرعة، والموثوقية، والبساطة، وهذا ما يجعلها مرغوبة للشركات الناشئة ولذلك يُستخدم FastAPI على نطاق واسع، بدءًا من مشاريع الشركات الناشئة وصولًا إلى التطبيقات المؤسسية الضخمة.

❖ مقارنتها مع الأطر الأخرى (Django, Flask):

FastAPI و Flask:



الميزة	FastAPI	Flask
الأداء	عالي (دعم العمليات الغير متزامنة)	اقل (يدعم العمليات المتزامنة فقط)
سلامة أنواع البيانات (Type Safety)	يستخدم تلميحات الأنواع (Python type hints)	لا يوجد تحقق مدمج للأنواع
التحقق من البيانات (Data Validation)	مدمج باستخدام Pydantic	يحتاج مكتبات إضافية
التوثيق (Documentation)	يؤكد تلقائيًا (Swagger UI, ReDoc)	يتطلب إعداد يدوي
الدعم غير المتزامن	نعم (async/await)	لا يوجد دعم أصيل للـ async
سهولة التعلم	متوسطة	سهلة

أهم الفروقات بين Flask و FastAPI:

- ◀ FastAPI أسرع بحكم دعمها للعمليات الغير متزامنة، بينما flask هو يدعم المتزامن فقط بشكل تلقائي.
- ◀ يوفّر FastAPI التحقق التلقائي من الطلبات (Automatic Request Validation)، في حين يحتاج Flask إلى مكتبات إضافية مثل Marshmallow للقيام بعملية التحقق.
- ◀ يُعدّ Flask أبسط للاستخدام في المشاريع الصغيرة، لكنه يتطلب جهداً يدوياً أكبر عند التحقق من الطلبات (API Validation) وإعداد التوثيق (Documentation).

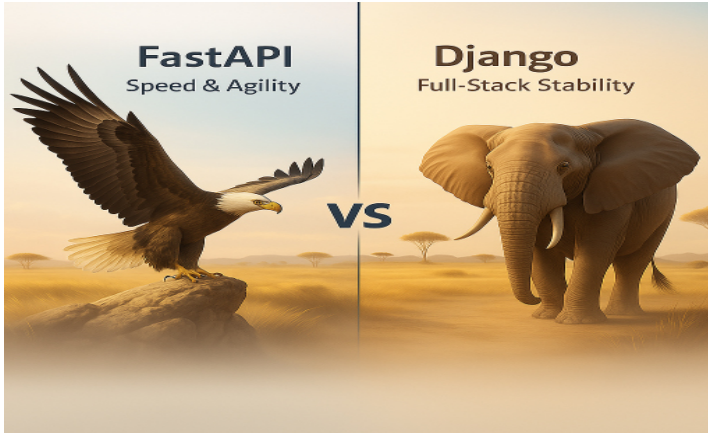
متى تستخدم Flask :

- ◀ عند بناء تطبيقات صغيرة أو واجهات برمجية بسيطة (Simple APIs).
- ◀ عند العمل مع فريق لديه خبرة سابقة في Flask ويعتمد على الإضافات (Extensions) الخاصة به.

متى تستخدم FastAPI :

- ◀ عندما يكون الأداء وقابلية التوسّع أمرًا مهمًا.
- ◀ عند بناء واجهات برمجية حديثة (Modern APIs) مع توثيق تلقائي (Automatic Documentation) والتحقق من البيانات بشكل تلقائي (automatic validation).
- ◀ عندما تعتمد بشكل كبير على البرمجة غير المتزامنة (Async-Heavy Workloads) مثل التطبيقات الآنية (Real-Time Applications) أو نماذج التعلّم الآلي (Machine Learning Models).

Django و FastAPI:



الميزة	FastAPI	Django
الأداء	عالي (دعم العمليات الغير متزامنة)	اقل (يدعم العمليات المتزامنة فقط)
سلامة أنواع البيانات (Type Safety)	يستخدم تلميحات الأنواع (Py-thon type hints)	لا يوجد تحقق مدمج للأنواع
التحقق من البيانات (Data Validation)	مدمج باستخدام Pydantic	يستخدم Django Forms و DRF Serializers
التوثيق (Documentation)	يولّد تلقائيًا (-Swagger UI, ReD-oc)	يتطلب إعداد يدوي
المزايا المدمجة	خفيف ومرن لا يوجد	يتضمن ORM، لوحة الإدارة بالنظام (Admin Panel)، مصادقة (-Authentication)، إلخ
سهولة التعلم	متوسطة	صعب
تركيز الإطار	إطار عمل موجّه لبناء الـ API	إطار ويب متكامل (Full-Stack Web Framework) بالدرجة الأولى

أهم الفروقات بين Django و FastAPI:

- ◀ Django هو إطار موجه لعمل ويب متكامل (Full-Stack Web Framework)، صُمم للتطبيقات التقليدية، ويتضمن ميزات مدمجة مثل: المصادقة (Authentication)، و ORM، ولوحة الإدارة في النظام (Admin Panel).
- ◀ FastAPI صُمم خصيصًا لبناء واجهات برمجية (APIs) و الخدمات المصغرة (Microservices).
- ◀ FastAPI أسرع بشكل ملحوظ لأنه مبني على التنفيذ غير المتزامن (Asynchronous Execution)، بينما Django (حتى مع Django REST Framework) يعمل بشكل أساسي بطريقة متزامنة (Synchronous).
- ◀ Django REST Framework خيار ممتاز للمطورين الذين يستخدمون Django بالفعل، لكنه يحتاج إلى إعداد إضافي من أجل توثيق الـ APIs ودعم البرمجة غير المتزامنة (Async Support).

متى تستخدم Django :

- ◀ عند بناء ويب متكامل (Full-Stack Applications) مع إدارة قواعد بيانات عن طريق الـ ORM) ومصادقة مدمجة.
- ◀ عند العمل على مشاريع تحتاج إلى لوحة إدارة (Admin Panel) ومنطق أعمال معقد (Complex Business Logic) في هيكل أحادي (Monolithic Structure).

متى تستخدم FastAPI :

- ◀ عند بناء خدمات مصغرة (Microservices) كنوع هيكلية نظام، و واجهات برمجية عالية الأداء (High-Performance APIs).
- ◀ عند الحاجة إلى معالجة غير متزامنة (Asynchronous Processing) للتعامل مع الطلبات واسعة النطاق بكفاءة.
- ◀ عند إعطاء الأولوية لـ التحقق التلقائي من البيانات (Automatic Validation)، والتوثيق (Docu-mentation)، والتطوير السريع للـ APIs.

والآن وبعد أن فهمنا الفروقات والمقارنة نقول FastAPI يُعدّ بديلاً ممتازاً لـ Flask في تطوير الـ APIs، كما يتفوق على Django عند بناء خدمات برمجية عالية الأداء (High-Performance) وقابلة للتوسع (Scalable). والآن، بعد أن تعمقنا في الجانب النظري، دعنا ننتقل إلى التطبيق العملي ونبحر معاً.

تمارين القسم:

- ▶ تعمّق أكثر في فهم آلية عمل العمليات المتزامنة (Synchronous) وغير المتزامنة (Asynchronous) [\(اقرأ\)](#).
- ▶ ولفهم ASGI و WSGI بشكل أوضح، لأننا سنتطرق إليهما لاحقاً في هذا الكتاب [\(اقرأ\)](#).



الوحدة | 2

ابدأ أول تطبيق لك باستخدام FastAPI

ابداً أول تطبيق لك باستخدام FastAPI

❖ متطلبات النظام وأفضل الممارسات:

قبل أن نبدأ في استخدام FastAPI، من المهم التأكد من أن بيئة التطوير (Development Environment) مهيأة لتلبية الاحتياجات، وتتبع أفضل الممارسات (Best Practices) من أجل تحسين الأداء (-Perfor-mance) وضمان قابلية الصيانة (Maintainability) مع مرور الوقت.

1. احتياجات النظام

◀ نسخة ال Python: يحتاج FastAPI إلى نسخة +Python 3.7, يفضل ذلك لدعم ميزة تلميحات الأنواع (Python type hints).

◀ التبعات (Dependencies):

- fastapi: الإطار
- Uvicorn: يحتاج FastAPI إلى خادم ASGI (مثل Uvicorn) لتشغيل تطبيقاته.
- Pydantic: تُستخدم للتحقق من البيانات (Data Validation).
- دعم قواعد البيانات (Database Support): يعمل مع قواعد بيانات SQL مثل (PostgreSQL, MySQL, SQLite, greSQL)، وكذلك قواعد بيانات NoSQL مثل (MongoDB)، وذلك باستخدام أطر ORM مثل SQLAlchemy و Tortoise-ORM، أو أطر ODM مثل ODMantic.

2. أفضل الممارسات لاستخدام fastAPI تنظيم الكود

هيكل مشروعك على شكل وحدات (Modules) لزيادة التنظيم وسهولة الصيانة

```
/app
  /routes
    user.py
    product.py
  /models
    user.py
    product.py
  database.py
  main.py
```

❖ تثبيت FastAPI و شغل أول تطبيق لك

قبل أن نبدأ في عملية البناء، دعنا نجهز البيئة الافتراضية (virtual environment) في بايثون، عند إنشاء مشروع جديد أو حتى عند العمل على مشروع قائم، نلجأ إلى استخدام البيئة الافتراضية لعزل تبعيات المشروع (dependencies) عن المشاريع الأخرى.

على سبيل المثال:

قد يحتاج مشروع أ إلى: fastapi==0.115.0 و uvicorn==0.23.0

بينما يحتاج مشروع ب إلى: fastapi==0.104.1 و uvicorn==0.21.0

لو عملنا على هذه المشاريع في البيئة العامة للنظام، ستحدث تعارضات بين الإصدارات، لكن باستخدام البيئة الافتراضية نستطيع إنشاء بيئة مستقلة لكل مشروع على حدة، وهو ما سنقوم به الآن.

تأكد من أن لديك Python 3.7+ أو أحدث مثبتًا على جهازك.
للتحقق من الإصدار الحالي، افتح terminal أو command prompt ثم نفذ أحد الأوامر التالية:

```
python --version
```

أو

```
python3 --version
```

ملاحظة: في بعض أنظمة التشغيل (مثل Linux و macOS) قد تحتاج لإستخدام **python3** بدلاً من **py-thon** للتحقق من الإصدار.

إذا كان إصدار Python لديك متوافقًا، يمكنك المتابعة. انتقل أولاً إلى المجلد (directory) الذي تريد إنشاء المشروع بداخله، ثم شغل الأمر التالي في ال terminal أو ال command prompt:

انشاء بيئة افتراضية#

```
python -m venv .venv
```

يجب تفعيل البيئة الافتراضية (virtual environment) في كل مرة تعمل فيها على المشروع.
إذا لم تقم بتفعيلها، فلن يتعرف المشروع على المكتبات التي ثبتتها داخله، وسيبدو وكأنك لم تقم بتثبيت أي شيء.

شغله

On Linux/Mac:

```
source venv/bin/activate
```

On Windows:

```
venv\Scripts\activate
```

والآن سوف نقوم بتثبيت FastAPI و Uvicorn باستخدام أداة إدارة الحزم الخاصة ببايثون pip:

```
pip install fastapi uvicorn
```

FastAPI: هو الإطار الأساسي لبناء واجهات برمجية (APIs)، حيث يتيح لنا تعريف المسارات (Routes)، التعامل مع الطلبات (Requests)، وإرجاع الاستجابات (Responses).

Uvicorn: خادم ASGI سريع يقوم بتشغيل تطبيق FastAPI، ويعمل كواجهة (Interface) بين التطبيق والويب.

وبمجرد اكتمال عملية التثبيت، ستكون جاهزاً لبدء بناء أول تطبيق لك باستخدام FastAPI.

❖ تشغيل أول واجهة برمجية (API) والوصول إليها عبر المتصفح (Browser)

الآن بعد أن قمنا بتثبيت FastAPI و Uvicorn، لننشئ تطبيقًا بسيطًا باستخدام FastAPI:

1. إنشاء ملف Python جديد
أنشئ ملفًا جديدًا باسم main.py (أو أي اسم آخر تفضله).
2. كتابة أول تطبيق FastAPI
افتح ملف main.py في محرر الأكواد لديك، وأضف الكود التالي:

```
from fastapi import FastAPI

# أنشئ كائن (instance) من الـ FastAPI class

app = FastAPI()

# أنشئ كائن (instance) من الـ FastAPI class
@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

شرح الكود:

- ◀ نبدأ باستيراد FastAPI من حزمة fastapi.
- ◀ نقوم بإنشاء كائن (instance) من الصنف (class) FastAPI باسم app، وهو الذي يمثل تطبيقنا.
- ◀ نعرّف مسار (route) باستخدام المعرّف @app.get("/")، هذا المسار يستمع لطلبات HTTP GET على مسار (/).
- ◀ الدالة read_root() هي المعالج (handler) لهذا المسار، عند الوصول إلى العنوان الجذري (route)، تُعيد (dictionary) يحتوي على رسالة:

```
{"message": "Hello, FastAPI!"}
```

تشغيل التطبيق باستخدام Uvicorn

لتشغيل التطبيق، افتح الـ terminal أو command prompt وانتقل إلى المجلد (directory) الذي يحتوي على ملف `main.py`.

بعد ذلك، شغل الأمر التالي لتشغيل تطبيقك بـ Uvicorn:

```
uvicorn main:app --reload
```

main: يشير إلى ملف بايثون (`main.py`). <

app: يشير إلى كائن FastAPI (الـ instance) المعرف داخل الملف. <

--reload: يفعّل ميزة إعادة تحميل (auto-reloading) الخادم تلقائيًا عند إجراء أي تغييرات في الكود، وهذه الميزة مفيدة جدًا أثناء التطوير. الوصول إلى الـ API من المتصفح <

بعد تشغيل التطبيق، افتح المتصفح وانتقل إلى الرابط التالي:

```
http://127.0.0.1:8000
```

ستظهر لك الاستجابة (JSON response) التي يرجعها تطبيق FastAPI:

```
{"message": "Hello, FastAPI!"}
```

❖ استخدام Swagger UI و ReDoc لتوثيق الـ API

يوفر FastAPI توثيقًا تفاعليًا للـ API بشكل تلقائي، هذه الميزة تجعل من السهل جدًا اختبار نقاط النهاية الخاصة (Endpoints) بالـ API، وفهم بنيتها، ورؤية الطلبات والاستجابات التي تتوقعها.

3. Swagger UI

للوصول إلى Swagger UI، انتقل ببساطة إلى الرابط التالي في المتصفح أثناء تشغيل الخادم:

```
http://127.0.0.1:8000/docs
```

في Swagger UI سترى تمثيلًا بصريًا للـ API الخاص بك، سيتم عرض المسار / مع طريقة GET، يمكنك النقر على زر "Try it out" ثم تنفيذ الطلب مباشرة من واجهة التوثيق، سيعرض لك Swagger UI الاستجابة (response) ويسمح لك بالتفاعل مع الـ API دون الحاجة إلى كتابة أي كود.

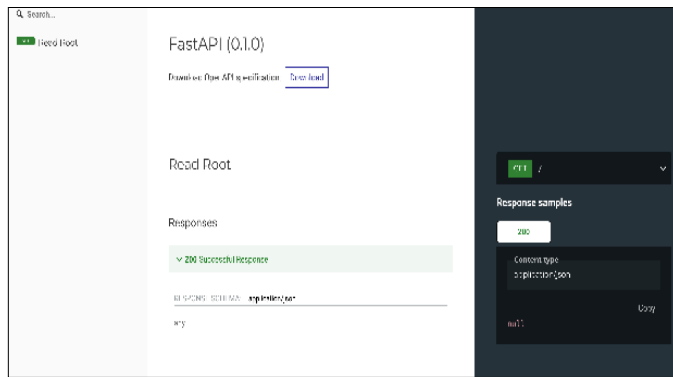
Code	Details
200	Response body <pre>{ "message": "Hello, FastAPI" }</pre> Response headers <pre>content-length: 28 content-type: application/json date: Mon, 29 Sep 2025 09:41:37 GMT server: uvicorn</pre>

2. ReDoc

يوفر FastAPI أيضًا واجهة توثيق مبنية على ReDoc، وهي طريقة أخرى تفاعلية وسهلة الاستخدام لاستكشاف الـ API الخاص بك. للوصول إلى ReDoc، انتقل إلى:

```
http://127.0.0.1:8000/redoc
```

يوفر ReDoc عرضًا أكثر تفصيلًا لتوثيق الـ API الخاص بك، مع شريط جانبي سهل التنقل وتصميم أنظف وأكثر تنظيمًا. وهو مفيد بشكل خاص مع الـ APIs المعقدة التي تحتوي على العديد من نقاط النهاية (endpoints) والوصف التفصيلي.



تخصيص التوثيق (Customizing the Documentation)

يمكنك إضافة بيانات تعريفية (metadata) مثل العنوان (title)، الوصف (description)، والإصدار (version-) (sion) للـ API الخاص بك. على سبيل المثال:

```
from fastapi import FastAPI

app = FastAPI(
    title="My First FastAPI App",
    description="This is a simple FastAPI application for learning purposes.",
    version="1.0.0",
)
```

سيؤدي ذلك إلى تعديل العنوان والوصف المعروضين في كل من Swagger UI و ReDoc، مما يجعل توثيق الـ API أكثر احترافية وتعبيراً عن تفاصيله، الآن، اذهب مرة أخرى إلى `/redoc` أو `/docs`، وستلاحظ أنَّ العنوان والوصف في أعلى التوثيق قد تغيرا وفقاً لما أضفته في الإعدادات.

❖ أساسيات بناء الـ APIs

في هذا القسم، سيتم استعراض المفاهيم الأساسية لبناء واجهات برمجية (APIs) باستخدام FastAPI. يتضمن ذلك تعريف المسارات (Routes) للتعامل مع مختلف طرق بروتوكول HTTP (GET, POST, PUT, DELETE)، بالإضافة إلى العمل مع المعاملات الديناميكية في المسار (Dynamic Path Parameters)، والتفريق بين معاملات الاستعلام (Query Parameters) و معاملات المسار (Path Parameters).

تعريف المسارات (Routes) لطرق HTTP: GET, POST, PUT, DELETE

يُسهّل FastAPI عملية تعريف المسارات (Routes) من خلال [Decorators](#) تتوافق مع طرق بروتوكول HTTP مثل GET و POST و PUT و DELETE. حيث تُستخدم كل طريقة لغرض محدد عند التفاعل مع الخادم (Server)، اريدك أن تجرب كل بروتوكول بنفسك لذلك في نفس الملف الذي أنشأناه مسبقاً يمكنك ان تكتب الاكواد القادمة فيه وتجربها وتشغلها بنفس الطريقة.

1. GET Method

تُستخدم طريقة **GET Method** لجلب المعلومات من الخادم (Server)، وهي الطريقة الأكثر شيوعاً للحصول على البيانات، في إطار عمل FastAPI يتم تعريف مسار (Route) خاص بـ GET باستخدام الـ `@app.get()` Decorator.

مثال:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

في هذا المثال، سيقبل المسار `/items/{item_id}` الطلبات من نوع GET ويُرجع كائناً (Instance) بصيغة JSON يحتوي على قيمة `item_id` المُمرّرة في الرابط (URL)، إذا مرت 3 ستحصل على اجابة بهذا الشكل `"item_id": 3`.

2. POST Method

تُستخدم **POST Method** لإرسال البيانات إلى الخادم (Server) بهدف إنشاء موارد جديدة، وتُطبق عادةً عند إرسال بيانات النماذج (Form Data)، أو عند إنشاء سجلات جديدة في قاعدة البيانات، أو عند إرسال بيانات (Payloads) إلى واجهة برمجية (API). في إطار العمل FastAPI يتم تعريف مسار (Route) خاص بـ POST باستخدام الـ `@app.post()` Decorator.

مثال:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# تعريف نموذج البيانات باستخدام Pydantic
class Item(BaseModel):
    name: str
    price: float

# تعريف مسار POST لإنشاء عنصر جديد
@app.post("/items/")
def create_item(item: Item):
    return {"name": item.name, "price": item.price}
```

في هذا المثال، يتوقع FastAPI أن يحتوي الطلب على جسم (Body) بصيغة JSON، حيث يتم تحليله إلى نموذج (model) Item (وهو نموذج مُعرّف باستخدام Pydantic) وسنتعرف عليه بشكل أعمق لاحقاً، ستقوم الـ API بإنشاء عنصر جديد وإرجاع استجابة تحتوي على اسم العنصر وسعره.

3. PUT Method

تُستخدم PUT Method لتحديث مورد موجود مسبقاً، حيث تقوم باستبدال التمثيل الحالي للمورد بآخر جديد، في إطار العمل FastAPI يتم تعريف مسار (Route) خاص بـ PUT باستخدام الـ Decorator `@app.put`

مثال:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# تعريف نموذج البيانات باستخدام Pydantic
class Item(BaseModel):
    name: str
    price: float

# تعريف مسار PUT لتحديث عنصر موجود
@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item):
    return {
        "item_id": item_id,
        "name": item.name,
        "price": item.price
    }
```

في هذا المثال، تستقبل الدالة `update_item` كلاً من معامل المسار (`item_id`) و جسم الطلب (`item`) ثم ومن ثم يتم طلب (`request`) ما نريد تحديثه و نقوم بإرجاع تفاصيل العنصر بعد تحديثه، شغل الكود كما تعلمنا سابقاً ثم اذهب الى <http://127.0.0.1:8000/docs> هناك تستطيع تجربة تغيير العنصر الذي انشأته من خلال POST .

4. DELETE Method

تُستخدم **DELETE Method** لحذف مورد (`Resource`) من الخادم (`Server`)، في إطار العمل FastAPI يتم تعريف مسار (`Route`) خاص بـ DELETE باستخدام الـ `@app.delete` Decorator.

مثال:

```
from fastapi import FastAPI

app = FastAPI()

# تعريف مسار DELETE لحذف عنصر موجود
@app.delete("/items/{item_id}")
def delete_item(item_id: int):
    return {"message": f"Item {item_id} has been deleted"}
```

في هذا المثال، تستقبل الدالة `delete_item` معامل المسار (`item_id`) وتُرجع رسالة تؤكد أنه قد تم حذف العنصر.

❖ المعاملات الديناميكية في المسار (Dynamic Path Parameters)

يتيح لك FastAPI تعريف معاملات ديناميكية في المسار، وهي بمثابة متغيرات داخل الرابط (URL) يمكن ملؤها بقيم محددة عند إرسال الطلب، وتُستخدم معاملات المسار لالتقاط المعلومات مباشرةً من الرابط.

لتعريف معامل ديناميكي في المسار، يتم وضع اسم المعامل داخل أقواس معقوفة {} ضمن مسار الرابط. بعد ذلك، يتم تمرير هذه القيم إلى الدالة كوسائط (Arguments).

مثال:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

في المثال أعلاه، يتم التقاط المعامل `item_id` من الـ URL مباشرةً على سبيل المثال، إذا تم إرسال طلب إلى `items/123/`، فإن FastAPI ستستدعي الدالة `read_item` مع `item_id=123`.

استخدام Dynamic Path Parameters يتيح لك إنشاء Endpoints مرنة وديناميكية، مفيدة عندما تريد استهداف مورد محدد بناءً على معرفه (Identifier)، مثل جلب، تحديث، أو حذف سجل معين.

❖ معاملات المسار (Path Parameters) و معاملات الاستعلام (Query Parameters)

يدعم FastAPI كلاً من معاملات الاستعلام (Query Parameters) ومعاملات المسار (Path Parameters) والتي تتيح لك تمرير البيانات إلى الخادم بطريقة منظمة.

معاملات المسار

كما ذكرنا سابقاً فإن معاملات المسار (Path Parameters) هي جزء من الرابط (URL) نفسه، ويتم تعريفها داخل الأقواس المعقوفة {} وغالباً ما تُستخدم هذه المعاملات لتحديد موارد معينة.

على سبيل المثال، لجلب تفاصيل عنصر محدد:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

يتم تمرير قيمة item_id مباشرة في الرابط (URL). على سبيل المثال، فإن طلباً إلى items/42/ سيعمرر القيمة 42 ك item_id إلى الدالة (Function).

معاملات الاستعلام

معاملات الاستعلام هي معاملات اختيارية يتم تمريرها في الرابط (URL) بعد علامة الاستفهام ?. إذا وُجد أكثر من معامل استعلام، يتم فصلها باستخدام علامة &. غالباً ما تُستخدم معاملات الاستعلام في عمليات التصفية (Filtering)، الترتيب (Sorting)، أو تجزئة النتائج (Pagination).

في FastAPI، يمكن تعريف معاملات الاستعلام (Query Parameters) من خلال إضافتها كوسائط (Ar-guments) للدالة (Function) مع تحديد نوع البيانات (Type Hints). يقوم FastAPI باستخراج هذه

مثال باستخدام معاملات الاستعلام:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/")
def read_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```



في هذا المثال، يقبل المسار /items/ معاملين من نوع Query Parameters هما: skip و limit. وإذا لم يتم تمرير قيم لهذين المعاملين، فإن قيمتهما الافتراضية ستكون 0 و 10.

قد يبدو الطلب (Request) على النحو التالي :

```
GET /items/?skip=0&limit=10
```

في هذه الحالة، سيقوم FastAPI بتمرير القيم skip=0 و limit=10 إلى الدالة read_items. وتُعد معاملات الاستعلام (Query Parameters) مفيدة جداً في الحالات التي تكون فيها المعاملات اختيارية، أو عند الحاجة إلى ضبط سلوك الـ API بدقة، مثل: تصفية قائمة من العناصر (Filtering) أو تجزئة النتائج (Pagination).

الجمع بين معاملات المسار (Path Parameters) ومعاملات الاستعلام (Query Parameters)

يمكنك استخدام معاملات المسار (Path Parameters) ومعاملات الاستعلام (Query Parameters) معاً في نفس المسار (Endpoint)، يقوم FastAPI بمعالجة كلا النوعين بسهولة، حيث يتم استخراجها من الطلب وتمريرها مباشرةً إلى الدالة.

مثال:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int, detail: bool = False):
    if detail:
        return {"item_id": item_id, "detail": "Full item details"}
    return {"item_id": item_id}
```

في هذا المثال، يُعتبر `item_id` معامل مسار (Path Parameter)، بينما يُعتبر `detail` معامل استعمال (Query Parameter).

سيكون الطلب:

```
GET /items/42?detail=true
```

سُرجع تفاصيل العنصر كاملة.

بينما طلب مثل:

```
GET /items/42
```

سُرجع فقط قيمة `item_id`.

باختصار، يوفر FastAPI طريقة سهلة وقوية لبناء واجهات برمجية (APIs) من خلال تعريف المسارات (Routes) باستخدام طرق بروتوكول HTTP Methods المختلفة (GET, POST, PUT, DELETE)، واستخدام معاملات المسار (Path Parameters) لالتقاط القيم مباشرةً من الرابط (URL)، ومعاملات الاستعلام (Query Parameters) لإضافة مرونة أكبر مثل Filtering (التصفية)، Sorting (الترتيب)، أو Pagination (تجزئة النتائج)، ومع خاصية التحقق التلقائي (Validation) من صحة البيانات وتحويل أنواعها (Type Conversion)، يضمن FastAPI أن تكون واجهاتك البرمجية فعّالة (Efficient) وموثوقة (Reliable) في آن واحد، إن إتقان هذه المفاهيم الأساسية يمكّنك من تصميم APIs سهلة الاستخدام (Easy to use)، عالية المرونة (Flexible)، وقادرة على التعامل مع مجموعة واسعة من حالات الاستخدام (Cases).



الوحدة 3

التحقق من صحة البيانات باستخدام Pydnatic

التحقق من صحة البيانات باستخدام Pydnatic

❖ بناء نماذج البيانات في FastAPI

في FastAPI، يُعد التحقق من صحة البيانات (Data Validation) جانبًا أساسيًا في بناء واجهات برمجية (APIs) موثوقة وأمنة حيث يدمج FastAPI مكتبة Pydantic، وهي مكتبة قوية مخصصة للتحقق من البيانات (Validation) وتحليلها (Parsing) للتعامل مع عملية التحقق من صحة البيانات الواردة إلى النظام.

تتيح لك Pydantic تعريف نماذج بيانات (Data Models) تضمن سلامة وصحة البيانات قبل استخدامها داخل التطبيق في هذا القسم، سنستعرض كيفية إنشاء واستخدام نماذج Pydantic، والتحقق من البيانات الواردة، ومعالجة الأخطاء، والتعامل مع الخصائص الافتراضية (Default Attributes) والخصائص المطلوبة (Required Attributes) داخل النماذج.

❖ استخدام Pydantic لإنشاء النماذج

توفر مكتبة Pydantic طريقة لتعريف نماذج البيانات (Data Models) باستخدام التعريفات النمطية (Type Annotations) في لغة Python تكون هذه النماذج عبارة عن فئات (Classes) تُحدد بنية البيانات المتوقعة سواء داخل جسم الطلب (Request Body) أو ضمن معاملات الاستعلام (Query Parameters). تقوم Pydantic تلقائيًا بالتحقق من البيانات (Data Validation) ومقارنتها بالنموذج المُعرّف، وتُرجع تنبيهًا واضحًا في حال لم تتطابق البيانات مع النوع (Type) أو البنية (Structure) المطلوبة.

لإنشاء نموذج (Pydantic Model)، يتم تعريف فئة (Class) ترث من `pydantic.BaseModel`. داخل هذه الفئة، يتم تحديد حقول النموذج (Fields) باستخدام تعريفات الأنواع (Type Annotations) في بايثون.

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    description: str = None
    tax: float = None
```

في هذا المثال، يعرف نموذج `Item` أربعة حقول:

- name: حقل نصي (String) اجباري.
- price: حقل رقمي عشري (Float) اجباري.
- description: حقل نصي (String) اختياري، قيمته الافتراضية `None`.
- tax: حقل عشري (Float) اختياري، قيمته الافتراضية `None` أيضًا.

يقوم Pydantic تلقائيًا بالتحقق من أن الحقل `name` يحتوي على نص (`string`)، وأن الحقل `price` يحتوي على عدد عشري (`float`) عند تمرير البيانات إلى هذا النموذج، وفي حال لم تتطابق البيانات الواردة مع الأنواع المتوقعة، فسيقوم Pydantic بإصدار خطأ تحقق (`Validation Error`) لضمان معالجة البيانات الصحيحة والمنظمة فقط داخل التطبيق.

❖ التحقق من البيانات ومعالجة الأخطاء

تُعد قدرة Pydantic على التحقق من صحة البيانات (Data Validation) واحدة من أهم ميزاتها يقوم FastAPI بالتحقق تلقائيًا من البيانات بمقارنتها مع النماذج (Models) التي تقوم بتعريفها، سواء عند التعامل مع جسم الطلب (Request Body) في طلبات POST و PUT و PATCH، أو مع معاملات الاستعلام (Query Parameters) وفي حال لم تتطابق البيانات مع الأنواع أو البنية المتوقعة، يقوم FastAPI تلقائيًا بإرجاع استجابة خطأ (Error Response) تحتوي على معلومات تفصيلية توضح سبب الخطأ وموقعه.

مثال على التحقق من البيانات (Validation Example):

دعنا نعرف مسارًا (Endpoint) يستخدم نموذج Item الذي أنشأناه سابقًا.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    description: str = None
    tax: float = None

@app.post("/items/")
async def create_item(item: Item):
    return {"name": item.name, "price": item.price}
```

الآن، إذا أرسلنا طلب POST يحتوي على بيانات لا تطابق البنية المتوقعة، فسيقوم FastAPI بإرجاع خطأ تلقائيًا، حاول معي وتبع افتح <http://127.0.0.1:8000/docs>، بعد تشغيل الكود بـ `uvicorn main:app --reload` واضغط على try it out واكتب بال request body

على سبيل المثال، إذا تم إرسال الحقل price كنص (string) بدلاً من رقم عشري (float):

```
{
  "name": "Laptop",
  "price": "not_a_number"
}
```

FastAPI سُرّجع خطأً من نوع Unprocessable Entity 422 يتضمن تفاصيل توضح المشكلة في التحقق من البيانات (Validation Error).

ستكون الاستجابة كما يلي:

```
{
  "detail": [
    {
      "type": "float_parsing",
      "loc": [
        "body",
        "price"
      ],
      "msg": "Input should be a valid number, unable to parse string as a number",
      "input": "not_a_number"
    }
  ]
}
```

تُوضح رسالة الخطأ هذه بوضوح أن الحقل price يجب أن يحتوي على قيمة صحيحة من نوع float. كما تُظهر موقع الخطأ بدقة (body -> price) انظر للسهم.

يساعد هذا المستوى من التفاصيل المطوّرين على تحديد الأخطاء بسرعة وتصحيح البيانات غير الصحيحة قبل معالجتها في النظام.

معالجة التحقق المخصّص (Handling Custom Validation):

يمكنك أيضًا إضافة منطق تحقق مخصّص (Custom Validation Logic) باستخدام Decorators من Pydantic مثل validator@ أو root_validator@ . تُستخدم لتنفيذ فحوصات إضافية أو لتعديل البيانات قبل إرجاعها.

على سبيل المثال:

إذا أردت التأكد من أن الحقل tax لا يمكن أن يكون سالبًا:

```
from pydantic import BaseModel, validator

class Item(BaseModel):
    name: str
    price: float
    tax: float = 0

    @validator("tax")
    def tax_must_be_positive(cls, value):
        if value < 0:
            raise ValueError("Tax must be a positive number")
        return value
```

في هذه الحالة، إذا أرسلنا عنصرًا يحتوي على قيمة سالبة في الحقل tax:

```
{
  "name": "Laptop",
  "price": 1000,
  "tax": -50
}
```

سُرجع FastAPI استجابة خطأ مشابهة لما يلي:

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "tax"
      ],
      "msg": "Value error, Tax must be a positive number",
      "input": -50,
      "ctx": {
        "error": {}
      }
    }
  ]
}
```


❖ الحقول الافتراضية (Default Fields) والحقول الإلزامية (Required Fields)

في Pydantic، يمكنك تعريف كل من الحقول الإلزامية (Required Fields) والحقول الاختيارية (Optional Fields).

الحقول الإلزامية هي الحقول التي يجب تضمينها في جسم الطلب (Request Body)، أما الحقول الاختيارية فيمكن عدم إرسالها، حيث ستحصل تلقائيًا على قيمة افتراضية (Default Value) والتي يمكن أن تكون None أو أي قيمة أخرى تقوم بتحديددها.

الحقول الإلزامية (Required Fields):

الحقل الإلزامي هو ببساطة الحقل الذي لا يحتوي على قيمة افتراضية (Default Value). بشكل افتراضي، جميع الحقول التي يتم تعريفها داخل نموذج Pydantic بدون قيمة افتراضية تُعتبر حقولًا إلزامية.

مثال:

```
class Item(BaseModel):
    name: str      # إلزامي
    price: float   # إلزامي
```

في هذا المثال، يُعتبر كل من name و price حقولًا إلزامية (Required Fields). إذا تم إغفال أحد هذين الحقلين من جسم الطلب (Request Body)، فسيُرجع FastAPI تلقائيًا خطأ تحقق (Validation Error) يُشير إلى أن البيانات المطلوبة مفقودة.

الحقول الاختيارية ذات القيم الافتراضية (Optional Fields with Defaults):

يمكنك تحديد الحقول الاختيارية (Optional Fields) من خلال تعيين قيمة افتراضية (Default Value) لها. إذا كان الحقل اختياريًا، فلن يتسبب في خطأ تحقق (Validation Error) في حال لم يتم تضمينه في جسم الطلب (Request Body).

مثال:

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    description: str = None
    tax: float = 0.0
```

في هذا المثال:

- ◀ الحقل description اختياري، وستكون قيمته الافتراضية None في حال عدم إرسالها.
- ◀ الحقل tax أيضًا اختياري، وستكون قيمته الافتراضية 0.0 في حال عدم إرسالها.

يُتيح ذلك لواجهة الـ API قبول الطلبات حتى في حال غياب هذه الحقول الاختيارية، مع الحفاظ على سلوك افتراضي واضح ومتوقع للبيانات.

استخدام الدالة Field() لتحديد القيم الافتراضية والقيود (Constraints):

يمكنك استخدام الدالة Field() من مكتبة Pydantic لإضافة تحقق إضافي (Additional Validation) على الحقول داخل النموذج، مثل تحديد القيم الدنيا والعليا (Minimum / Maximum Values)، أو فرض قيود على الطول (Length Constraints)، أو استخدام التعبيرات النمطية (Regular Expressions - Regex) للتحكم في شكل القيم المسموح بها حيث تُتيح لك هذه الطريقة إنشاء قواعد تحقق دقيقة ومرنة مباشرة داخل نموذج البيانات (Data Model).

مثال:

```
from pydantic import BaseModel, Field

class Item(BaseModel):
    name: str = Field(..., min_length=3) # حقل إلزامي، الحد الأدنى للطول 3 أحرف
    price: float = Field(..., ge=0) # حقل إلزامي، القيمة يجب أن تكون  $\geq 0$ 
    description: str = None # حقل اختياري
    tax: float = Field(0.0, ge=0) # حقل اختياري، القيمة الافتراضية 0.0، ويجب أن تكون  $\geq 0$ 
```

في هذا المثال:

- الحقل name إلزامي، ويجب أن يحتوي على 3 أحرف على الأقل كحد أدنى.
- الحقل price إلزامي، ويجب أن تكون قيمته أكبر من أو تساوي 0.
- الحقل tax يحتوي على قيمة افتراضية (0.0) ويجب أيضًا أن تكون أكبر من أو تساوي 0.

إذا فشل التحقق على سبيل المثال، إذا كانت قيمة price سالبة أو كان name أقصر من 3 أحرف فسيقوم FastAPI تلقائيًا بإرجاع خطأ تحقق (Validation Error) يحتوي على تفاصيل دقيقة توضح الحقول غير الصالحة وسبب الخطأ.

إلى الآن في هذا القسم تعلمنا كيفية إنشاء نماذج بيانات (Data Models) في FastAPI باستخدام Pydantic، وتناولنا ما يلي:

- كيفية تعريف النماذج باستخدام التعريفات النمطية في بايثون (Python Type Annotations).
- التحقق التلقائي من صحة البيانات باستخدام FastAPI و Pydantic.
- معالجة أخطاء التحقق (Validation Errors) من خلال رسائل خطأ واضحة ومفصلة.
- تعريف الحقول الإلزامية (Required Fields) والحقول الاختيارية (Optional Fields)، وتحديد القيم الافتراضية (Default Values) والقيود (Constraints) على الحقول.

يُعد التحقق من البيانات (Data Validation) عنصرًا أساسيًا لضمان أن يعمل تطبيقك كما هو متوقع، وأن يتعامل بسلاسة مع المدخلات غير الصحيحة وبفضل Pydantic، يوفر FastAPI نظامًا قويًا وسهل الاستخدام يضمن سلامة البيانات (Data Integrity) داخل تطبيقك، مما يسمح لك بالتركيز على بناء المنطق بدلاً من الانشغال بمعالجة التحقق والأخطاء يدويًا.

❖ التعامل مع الطلبات والاستجابات

في FastAPI، يُعَد التعامل مع الطلبات (Requests) والاستجابات (Responses) جزءًا أساسيًا من دورة حياة واجهة الـ API، يوفر FastAPI دعمًا قويًا لتحليل البيانات الواردة (Parsing Incoming Data) وتخصيص الاستجابات (Customizing Responses) ومعالجة الأخطاء (Error Handling) بشكل فعال ومنظم في هذا القسم، سنستعرض كيفية معالجة البيانات بصيغة JSON الواردة إلى النظام، وكيفية تخصيص الاستجابات (Responses)، بالإضافة إلى استخدام HTTPException لإدارة الأخطاء بكفاءة واحترافية.

تحليل البيانات الواردة بصيغة JSON

يدعم FastAPI تحليل البيانات الواردة (Parsing Incoming Data) بعدة صيغ، وتُعد صيغة JSON من أكثر الصيغ استخدامًا في واجهات الويب البرمجية (Web APIs)، عند إرسال العميل (Client) بيانات بصيغة JSON إلى الخادم (Server)، يقوم FastAPI تلقائيًا بتحويل هذه البيانات إلى كائنات بايثون (Python Objects) يمكن استخدامها داخل نقاط النهاية (Endpoints)، تعتمد هذه العملية على نماذج (Pydantic Models) التي تقوم بالتحقق من صحة البيانات (Validation) والتأكد من مطابقتها للبنية المحددة قبل أن تتم معالجتها في التطبيق.

استخدام نماذج Pydantic لتحليل (Parse) بيانات JSON:

يقوم FastAPI تلقائيًا بتحليل بيانات JSON الواردة وتحويلها إلى كائنات Python باستخدام نماذج البيانات (Data Models) التي تُعرّف باستخدام Pydantic، عندما يرسل العميل طلبًا من نوع POST أو PUT أو PATCH يحتوي على JSON body، يقوم FastAPI بقراءة بيانات JSON ومحاولة مطابقتها مع حقول النموذج (Model Fields) بناءً على أنواعها المحددة (Type Annotations).

على سبيل المثال، تأمل النموذج (model) التالي:

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    description: str = None
    tax: float = None
```

الآن يمكنك استخدام هذا النموذج لتحليل بيانات JSON الواردة في أحد المسارات (Route) داخل التطبيق:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    description: str = None
    tax: float = None

@app.post("/items/")
async def create_item(item: Item):
    return {"name": item.name, "price": item.price}
```



إذا أرسل العميل (Client) طلبًا من نوع POST مثل المثال التالي:

```
{
  "name": "Laptop",
  "price": 1000.0
}
```

سيقوم FastAPI تلقائيًا بتحليل (Parse) محتوى الطلب (Request Body) وتحويله إلى كائن (Instance) من نموذج Item، وسيحتوي المتغير item في الدالة create_item على هذا الكائن، بحيث يمكن الوصول إلى خصائصه تمامًا كما تصل إلى خصائص أي كائن Python عادي.

معالجة الأخطاء أثناء عملية التحليل (Parsing Error Handling):

إذا لم تتطابق بيانات JSON الواردة مع البنية المتوقعة، فسيُرجع FastAPI خطأً في التحقق (Validation Error). على سبيل المثال، إذا تم إرسال الحقل price كسلسلة نصية (string) بدلاً من رقم عشري (float)، فسيقوم FastAPI بإرجاع استجابة تحتوي على الخطأ التالي:

```
{
  "detail": [
    {
      "type": "float_parsing",
      "loc": [
        "body",
        "price"
      ],
      "msg": "Input should be a valid number, unable to parse string as a number",
      "input": "fsdfsdf"
    }
  ]
}
```

يشير هذا الخطأ إلى الموقع الدقيق للبيانات غير الصالحة (على سبيل المثال: price -> body) ويقدم رسالة واضحة توضح طبيعة المشكلة.

❖ تخصيص استجابات HTTP (Customizing HTTP Responses)

يمكنك FastAPI تحكّمًا كاملاً في بنية ومحتوى استجابات HTTP التي يتم إرجاعها من نقاط النهاية (Endpoints) يمكنك تخصيص كود الحالة (Status Code) والعناوين (Headers) ونوع المحتوى (Content-Type) وهيكّل الجسم (Body) بما يتناسب مع متطلبات تطبيقك.

تخصيص أكواد الحالة (Customizing Status Codes):

بشكل افتراضي، يُرجع FastAPI كود الحالة 200 OK عند نجاح طلبات GET و POST و PUT و PATCH ومع ذلك، يمكنك تحديد كود حالة مختلف للاستجابة باستخدام الوسيط (parameter) المسمى status_code داخل الـ route decorator.

مثال:

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/", status_code=status.HTTP_201_CREATED)
async def create_item(item: Item):
    return {"name": item.name, "price": item.price}
```

في هذا المثال، عند إنشاء العنصر بنجاح، سترجع FastAPI كود الحالة 201 Created بدلاً من القيمة الافتراضية 200 OK. يتيح لك الوسيط (status_code) تحديد كود حالة HTTP المناسب لكل حالة مختلفة مثل 404 Not Found عندما لا يتم العثور على مورد، أو 500 Internal Server Error عند حدوث خطأ في الخادم.

تخصيص محتوى الاستجابة (Customizing Response Content):

يمكنك تخصيص محتوى الاستجابة (Response Content) باستخدام الكلاسين JSONResponse أو PlainTextResponse لإرجاع بيانات بتنسيق JSON أو نص عادي (Plain Text) على التوالي، كما يتيح لك FastAPI إرجاع أنواع محتوى مخصصة (Custom Content Types) واستجابات معقدة (Complex Responses).

على سبيل المثال، قد ترغب في إرجاع استجابة JSON تحتوي على بنية مخصصة تتضمن تاريخًا زمنيًا (timestamp) أو بيانات وصفية إضافية (metadata) إلى جانب البيانات الأساسية:

مثال:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse
from pydantic import BaseModel
from datetime import datetime

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/")
async def create_item(item: Item):
    response_data = {
        "timestamp": datetime.now().isoformat(),
        "data": {
            "name": item.name,
            "price": item.price
        }
    }
    return JSONResponse(content=response_data)
```

في هذا المثال، يتم استخدام JSONResponse لتغليف بيانات الاستجابة (response data) مع تضمين الطابع الزمني (timestamp) ضمن الاستجابة. يتيح لك ذلك تخصيص بنية الاستجابة (-Response Struct) بالطريقة التي تناسب احتياجات تطبيقك.

العناوين المخصصة (Custom Headers):

يمكنك أيضاً تخصيص عناوين HTTP الخاصة بالاستجابة، فعلى سبيل المثال، قد ترغب في تعيين عناوين مثل Content-Type أو Cache-Control أو أي عنوان آخر ضمن الاستجابة، يمكنك تنفيذ ذلك من خلال تمرير العناوين (headers) باستخدام الوسيط (argument) المسمى headers داخل الاستجابة

مثال:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/custom_headers/")
async def custom_headers():
    headers = {"X-Custom-Header": "MyCustomHeaderValue"}
    return JSONResponse(
        content={"message": "Custom headers added!"},
        headers=headers
    )
```

سُيُرسَل هذا المثال استجابة تحتوي على رأس مخصص (Custom Header) باسم X-Custom-Header تم تعيين قيمته إلى MyCustomHeaderValue.

❖ استخدام HTTPException لمعالجة الأخطاء (ror Handling)

في FastAPI تُستخدم الكلاس HTTPException لمعالجة الأخطاء بطريقة سلسلة ومنظمة، تتيح لك هذه الكلاس رفع استثناءات (raise exceptions) تحتوي على أكواد حالة HTTP محددة ورسائل خطأ مخصصة (custom error messages) يُستخدم ذلك عادةً عندما لا تتحقق شروط معينة مثل عدم العثور على مورد (resource not found) أو وجود طلب غير صالح (invalid request).

رفع استثناء (Raising HTTPException):

يتم رفع استثناء HTTPException داخل دوال المسارات (route functions) باستخدام التعليمة raise، يمكنك تحديد كود الحالة (status code) ورسالة التفاصيل (detail message) الخاصة بالخطأ سيقوم FastAPI تلقائيًا بإرجاع استجابة (response) تحتوي على كود الحالة المحدد ورسالة الخطأ ضمن جسم الاستجابة (response body).

مثال على رفع استثناء عند عدم العثور على عنصر:

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

items = {"1": {"name": "Item 1", "price": 10}}

@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(
            status_code=404,
            detail="Item not found"
        )
    return items[item_id]
```

في هذا المثال، إذا طلب العميل (Client) عنصرًا غير موجود على سبيل المثال: items/2/ فسيقوم FastAPI برفع استثناء (HTTPException) بأكود الحالة 404 Not Found ورسالة التفاصيل "Item not found".

ستكون الاستجابة التي يتم إرجاعها بالشكل التالي:

```
{
  "detail": "Item not found"
}
```

إذا جربت الكود هذا ما سوف يظهر لك كنتيجة:

```
404
Undocumented
Error: Not Found
Response body
{
  "detail": "Item not found"
}
```

معالجة بحالات HTTP محددة (Handling Specific HTTP Status Codes):

يدعم HTTPException في FastAPI مجموعة من أكواد الحالة (HTTP Status Codes) المختلفة، مثل:

- ⏪ 404 Not Found عند عدم العثور على المورد.
- ⏪ 400 Bad Request عند إرسال بيانات غير صالحة.
- ⏪ 500 Internal Server Error عند حدوث مشكلة غير متوقعة في الخادم.
- ⏪ 401 Unauthorized عند فقدان بيانات المصادقة أو كونها غير صالحة.

يمكنك استخدام هذه الأكواد لإيصال نوع المشكلة بشكل دقيق وواضح إلى العميل (Client).

مثال على رفع استثناء (Bad Request 400) عند استلام بيانات غير صالحة:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/")
async def create_item(item: Item):
    if item.price <= 0:
        raise HTTPException(
            status_code=400,
            detail="Price must be greater than zero"
        )
    return {"name": item.name, "price": item.price}
```

إذا أرسل العميل (Client) سعرًا أقل من أو يساوي الصفر، فسيُرجع الخادم (Server) استجابة بأكود الحالة 400 Bad Request مع الرسالة: "Price must be greater than zero".

400 Error: Bad Request
Undocumented
Response body

```
{
  "detail": "Price must be greater than zero"
}
```

في ختام هذه الوحدة، نؤكد أن التعامل الفعّال مع الطلبات والاستجابات يُعدّ عنصرًا أساسيًا في بناء واجهات APIs قوية وسهلة الاستخدام، ويوفّر FastAPI مجموعة من الأدوات المتقدّمة التي تمكّنك من معالجة البيانات بكفاءة، والتعامل مع الأخطاء بسلاسة ووضوح، وتخصيص الاستجابات بما يتناسب مع احتياجات التطبيق، مما يساهم في إنشاء نقاط نهاية (API Endpoints) عالية الجودة واحترافية.



4 | الوحدة

تحسين الأداء باستخدام البرمجة غير المتزامنة

تحسين الأداء باستخدام البرمجة غير المتزامنة

❖ فهم آلية Async/Await في FastAPI

إحدى أبرز مزايا FastAPI هي دعمه المدمج للبرمجة غير المتزامنة (Asynchronous Programming)، تتيح البرمجة غير المتزامنة كتابة كود غير حاجز (Non-blocking Code) يمكنه التعامل مع مهام متعددة في الوقت نفسه، مما يجعل التطبيق أكثر كفاءة (Efficient) وقابلية للتوسع (Scalable).

في هذا القسم، سنتعمق في المفاهيم الأساسية للعمليات المتزامنة (Synchronous) وغير المتزامنة (Asynchronous)، ونتعلم كيفية استخدام الكلمة المفتاحية `async def` لتحسين الأداء في FastAPI، بالإضافة إلى كيفية تطبيق واجهات APIs غير متزامنة عمليًا.

مقارنة العمليات المتزامنة مقابل العمليات غير المتزامنة:

في البرمجة المتزامنة (Synchronous Programming)، يتم تنفيذ المهام واحدة تلو الأخرى، بحيث تحجب المهمة الحالية تنفيذ المهام التالية حتى تنتهي تمامًا وعلى الرغم من أن هذا النموذج بسيط وسهل الفهم، إلا أنه قد يؤدي إلى انخفاض في الكفاءة، خصوصًا عند التعامل مع المهام التي تتطلب الانتظار مثل استعلامات قواعد البيانات (Database Queries)، أو عمليات الإدخال والإخراج للملفات (File I/O)، أو الاتصال بواجهات APIs خارجية.

العمليات المتزامنة

في التطبيق المتزامن (Synchronous Application)، يتم تنفيذ الدالة أو المهمة بطريقة حازجة (Blocking)، أي أن البرنامج ينتظر اكتمال كل مهمة قبل الانتقال إلى المهمة التالية، قد يكون هذا الأسلوب مناسبًا للمهام البسيطة أو التطبيقات ذات عدد الطلبات القليل، لكنه يصبح غير فعال عندما تحتاج إلى التعامل مع عمليات متعددة طويلة المدى في الوقت نفسه.

مثال على دالة متزامنة (Synchronous Function):

```
import time

def fetch_data():
    time.sleep(2) # محاكاة عملية إدخال/إخراج بطيئة (مثل انتظار رد من قاعدة بيانات)
    return {"message": "Data fetched successfully!"}

def process_data():
    time.sleep(1) # محاكاة عملية أخرى تستغرق وقتًا (مثل معالجة البيانات)
    return {"message": "Data processed successfully!"}

def main():
    result1 = fetch_data()
    result2 = process_data()
    print(result1)
    print(result2)

main()
```

في الكود أعلاه، تُعدّ الدالتان `fetch_data()` و `process_data()` دالتين متزامنتين (Synchronous Functions). كل دالة تحجب تنفيذ البرنامج حتى تنتهي من تنفيذ مهمتها بالكامل. فإذا احتاج البرنامج إلى تنفيذ العديد من العمليات المماثلة، فإن إجمالي الوقت المستهلك في انتظار عمليات الإدخال/الإخراج (I/O) سيصبح كبيرًا، مما يؤدي إلى انخفاض الأداء العام للتطبيق.

العمليات غير المتزامنة

على النقيض من البرمجة المتزامنة، تتيح لك البرمجة غير المتزامنة (Asynchronous Programming) كتابة كود لا يحجب تنفيذ المهام الأخرى أثناء انتظار اكتمال العمليات التي تعتمد على الإدخال والإخراج (I/O-bound operations).

باستخدام البرمجة غير المتزامنة، يتم تنفيذ المهام التي تتطلب الانتظار مثل إجراء طلبات HTTP أو الاستعلام من قاعدة بيانات بالتوازي (concurrently) عندما نقول بالتوازي فهو تعبير لكن هي تعمل بآلية تشبه التوازي، مما يسمح للمهام الأخرى بالاستمرار في التنفيذ في الوقت نفسه، هذا يُعدّ مفيدًا جدًا للتطبيقات المعتمدة على عمليات الإدخال والإخراج (I/O-bound)، حيث يقضي البرنامج جزءًا كبيرًا من وقته في انتظار استجابات من أنظمة خارجية؛ في لغة Python، يتم تحقيق البرمجة غير المتزامنة باستخدام مكتبة `asyncio`، وتُستخدم الكلمتان `def` الافتتاحيتان `async` و `await` لتحريف وتنفيذ الدوال غير المتزامنة (Asynchronous Functions).

استخدام `async def` لتحسين الأداء:

يُسهّل FastAPI الاستفادة من البرمجة غير المتزامنة (Asynchronous Programming) من خلال استخدام الصياغة `async def` داخل دوال المسارات (Route Handlers)، فعند تعريف دوال Endpoints باستخدام `async def`، يستطيع FastAPI معالجة طلبات متعددة في الوقت نفسه دون حجب التنفيذ (Non-blocking Execution)، وهذا يؤدي إلى تحسين الأداء بشكل ملحوظ، خصوصًا في الحالات التي تتضمن عمليات إدخال/إخراج (I/O-bound) مثل استعلامات قواعد البيانات أو الاتصال بواجهات APIs خارجية. عند تعريف المسار باستخدام `async def`، يفهم FastAPI أن الدالة غير متزامنة، ويستخدم الكلمة المفتاحية `await` داخليًا لتنفيذ المهام التي تتطلب انتظارًا، مثل استدعاءات HTTP أو طلبات قاعدة البيانات دون تعطيل معالجة الطلبات الأخرى.

مثال أساسي على نقطة نهاية غير متزامنة:

لنأخذ مثالًا يُظهر كيفية تعريف مسار (Route) في FastAPI يقوم بتنفيذ مهمة غير متزامنة (Asynchronous Task).

هذا المثال يوضح كيف يمكن لـ FastAPI تنفيذ المهام دون حجب باقي الطلبات أثناء الانتظار:

```
from fastapi import FastAPI
import asyncio

app = FastAPI()

async def fake_db_query():
    await asyncio.sleep(2) # محاكاة استعلام قاعدة بيانات يستغرق ثانيتين
    return {"message": "Database query completed"}

@app.get("/async-example")
async def async_example():
    result = await fake_db_query() # انتظار نتيجة العملية غير المتزامنة
    return result
```

في هذا المثال:

تم تعريف الدالة `fake_db_query()` باستخدام `async def`، وتستخدم `await` `asyncio.sleep(2)` لمحاكاة تأخير مدته ثانيتين، كما يحدث عادةً عند استعلام قاعدة بيانات أو إرسال طلب إلى واجهة API خارجية.

المسار `async_example()` تم تعريفه أيضًا باستخدام `async def`، وينتظر نتيجة تنفيذ الدالة `fake_db_query()` باستخدام `await`.

على الرغم من أن `fake_db_query()` تحتوي على تأخير لمدة ثانيتين، إلا أن FastAPI يمكنه متابعة معالجة طلبات أخرى في الوقت نفسه أثناء انتظار النتيجة، مما يجعل التطبيق أكثر كفاءة (Efficient) وقابلية للتوسع (Scalable)، حيث يستطيع التعامل مع طلبات متعددة بشكل غير متزامن (Asynchronous).

الفوائد الرئيسية لاستخدام Async في FastAPI

عمليات إدخال/إخراج غير حازمة (Non-blocking I/O)

من خلال استخدام Async، يمكن لـ FastAPI التعامل مع عمليات الإدخال والإخراج (I/O-bound operations) مثل استعلامات قواعد البيانات أو طلبات HTTP بشكل متزامن (Concurrently)، وهذا يقلل الوقت الضائع في الانتظار، ويحسن سرعة استجابة واجهة الـ API بشكل ملحوظ.

قابلية التوسع العالية (Better Scalability)

تمكّن البرمجة غير المتزامنة تطبيقك من التوسع بسهولة للتعامل مع عدد أكبر من المستخدمين والطلبات دون الحاجة إلى تغييرات كبيرة في الكود فمن خلال معالجة عدة طلبات في الوقت نفسه، يمكن للنظام التعامل مع حجم مرور (Traffic) مرتفع دون زيادة في استهلاك الموارد.

الكفاءة في استخدام الموارد (Higher Efficiency)

تتيح آلية Async استخدام الموارد بشكل أكثر ذكاءً وكفاءة، حيث يمكن للتطبيق متابعة تنفيذ مهام أخرى أثناء انتظار مهمة واحدة مثل انتظار استجابة من شبكة خارجية، وهذا يؤدي إلى تحسين استغلال وحدة المعالجة المركزية (CPU) والذاكرة (Memory) وخاصة في التطبيقات التي تعتمد على عمليات الإدخال/

الإخراج (I/O-bound Applications).

مثال عملي على واجهة API غير متزامنة:

لنوسّع المثال السابق بسيناريو واقعي: نفترض أننا نبني واجهة API تتواصل مع خدمتين خارجيتين إحداهما لجلب بيانات المستخدم، والأخرى لاسترجاع سجلّ المعاملات. هذه العمليات قد تستغرق وقتًا، لذا فإن استخدام البرمجة غير المتزامنة يساعد على تحسين أداء الـ API.

```
from fastapi import FastAPI
import asyncio

app = FastAPI()

async def fetch_user_data(user_id: int):
    await asyncio.sleep(1) # محاكاة استعلام قاعدة بيانات أو نداء خارجي
    return {"user_id": user_id, "name": "John Doe"}

async def fetch_transaction_history(user_id: int):
    await asyncio.sleep(2) # محاكاة استعلام آخر أبطأ
    return {"user_id": user_id, "transactions": ["purchase1", "purchase2"]}

@app.get("/user/{user_id}")
async def get_user_info(user_id: int):
    user_data = await fetch_user_data(user_id)
    transaction_data = await fetch_transaction_history(user_id)
    return {**user_data, **transaction_data}
```

في هذا المثال:

تقوم الدالتان `fetch_user_data()` و `fetch_transaction_history()` بمحاكاة عمليات إدخال/إخراج غير متزامنة (Asynchronous I/O Operations) باستخدام `await asyncio.sleep()` لتمثيل الوقت المستغرق في جلب البيانات من خدمات خارجية.

المسار `get_user_info()` هو مسار غير متزامن (Asynchronous Route) يستدعي كلا من `fetch_user_data()` و `fetch_transaction_history()` وينتظرهما بشكل متزامن بالتوازي (Concurrently)، مما يتيح تنفيذ المهمتين في نفس الوقت دون حجب (Blocking).

وبما أن هذه الدوال غير متزامنة (Asynchronous)، فإن FastAPI يستطيع معالجة طلبات متعددة في الوقت نفسه أثناء انتظار استجابات الخدمات الخارجية، مما يؤدي إلى تحسين معدل الإنتاجية لأداء واجهة

الـ API.

يستغرق تنفيذ المسار `get_user_info` حوالي ثانيتين فقط، بينما كان سيستغرق حوالي 3 ثوانٍ لو كانت العمليات متزامنة (Synchronous)، وذلك لأن المهمتين تُنفَّذان بالتوازي (Concurrently) بدلاً من التتابع (Sequentially).

في ختام هذا الفصل، أدركنا كيف تُعدّ البرمجة غير المتزامنة (Asynchronous Programming) عاملاً محورياً في رفع كفاءة الأداء وقدرة التوسّع (Scalability) في تطبيقات FastAPI الحديثة.

ففي حين تُنفَّذ العمليات المتزامنة (Synchronous) بطريقة حازجة تُجبر النظام على انتظار كل مهمة حتى تكتمل، تتيح العمليات غير المتزامنة (Asynchronous) تنفيذ مهام متعددة بالتوازي، مما يحقق استغلالاً أمثل للموارد ويقلل من زمن الاستجابة.

ومن خلال استخدام الكلمة المفتاحية `async def` في تعريف دوال المسارات، يستطيع FastAPI التعامل مع عدد كبير من الطلبات بشكل متزامن دون تأخير أو حجب، مما يرفع من سرعة التطبيق وكفاءته، وقد رأينا عبر الأمثلة العملية كيف تُستخدم الدوال غير المتزامنة للتفاعل مع الخدمات الخارجية في الوقت نفسه، ما يؤدي إلى تحسين الأداء العام وتقليل وقت الانتظار.

إنّ إتقان استخدام `async` و `await` ليس مجرد تحسين برمجي، بل هو نهج هندسي متقدّم لبناء واجهات برمجة تطبيقات (APIs) سريعة، موثوقة، وقابلة للتوسّع وهي سمة جوهرية في تصميم الأنظمة الخلفية الحديثة التي تتطلب أداءً عاليًا واستجابة فورية في بيئات العمل كثيفة الطلبات.



الوحدة | 5

العمل مع قواعد البيانات

العمل مع قواعد البيانات

❖ الاتصال بقاعدة البيانات باستخدام SQLAlchemy

في تطبيقات الويب الحديثة، تلعب قواعد البيانات دورًا محوريًا في تخزين البيانات وإدارتها، وبما أن FastAPI إطار عمل مرّن وعالي الأداء، فإنه يتكامل بسلاسة مع قواعد بيانات SQL مثل SQLite و PostgreSQL، وتُعدّ SQLAlchemy واحدة من أكثر مكتبات ORM (Object Relational Mapper) شيوعًا في منظومة Python، إذ تتيح لك التفاعل مع قاعدة البيانات بأسلوب كائني التوجه (Object-Oriented)، مما يجعل عمليات قاعدة البيانات أكثر وضوحًا وأقل عرضة للأخطاء.

في هذا القسم، سنتعرّف على كيفية ربط FastAPI بقاعدة بيانات باستخدام SQLAlchemy، وتعريف نماذج الجداول (Database Models & Tables)، بالإضافة إلى إنشاء واجهة برمجة تطبيقات (API) لتنفيذ عمليات CRUD (إنشاء Create، قراءة Read، تحديث Update، حذف Delete) على قاعدة البيانات.

إعداد SQLite / PostgreSQL باستخدام SQLAlchemy :

تثبيت الحزم المطلوبة

للاتصال بقاعدة البيانات واستخدام SQLAlchemy مع FastAPI، نحتاج إلى تثبيت المكتبات الضرورية التالية:

```
pip install "fastapi[all]" sqlalchemy psycopg2-binary
```

❏ **fastapi[all]**: يقوم بتثبيت FastAPI مع مجموعة من الأدوات الإضافية المفيدة لتطوير تطبيقات Full-Stack.

❏ **sqlalchemy**: يقوم بتثبيت مكتبة SQLAlchemy، وهي أداة ORM (Object Relational Mapper) تساعد على التعامل مع قواعد البيانات بطريقة كائنية التوجه (Object-Oriented).

PostgreSQL database adapter for Py- لبايثون **psycopg2** هو محول PostgreSQL database adapter for Py- (thon)، إذا كنت تستخدم قاعدة بيانات SQLite، فلست بحاجة إلى تثبيت هذه الحزمة.

بعد تثبيت هذه الحزمة، يمكنك البدء في إعداد الاتصال بقاعدة البيانات (Database Connection Setup) في تطبيقك.

إعداد قاعدة بيانات SQLite :

تعد SQLite قاعدة بيانات خفيفة الوزن (Lightweight Database) وسهلة الإعداد، وتستخدم عادةً في بيئات التطوير والاختبار، تُخزن قاعدة بيانات SQLite في ملف واحد بسيط على نظامك، مما يجعلها خيارًا عمليًا وسريعًا لبدء العمل دون الحاجة إلى إعداد خادم قاعدة بيانات منفصل.

انشيء ملف database.py بنفس مستوى ال main.py بالتطبيق، وضع بداخله هذا الكود:

```
from sqlalchemy import create_engine

# SQLite بقاعدة بيانات
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

# SQLAlchemy (Engine) محرك
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}
)
```

في حالة SQLite، نستخدم (Connection URL) التالية:

sqlite:///./test.db

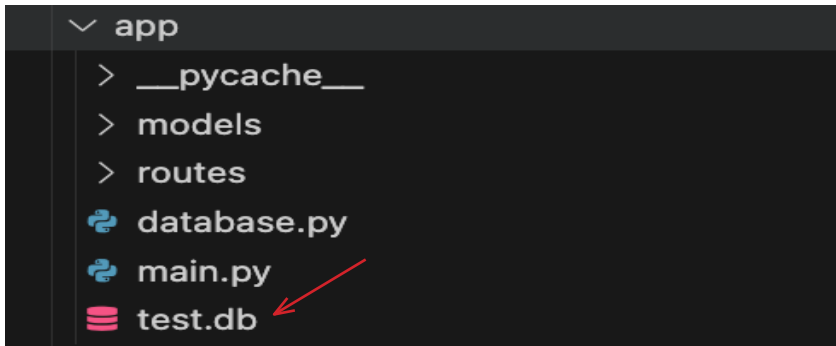
حيث يُمثل test.db اسم ملف قاعدة البيانات الخاص بـ SQLite، والذي سيتم إنشاؤه داخل المجلد الحالي للمشروع (./).

بهذه الطريقة، يتم تخزين قاعدة البيانات في ملف واحد محلي، مما يجعلها خيارًا مثاليًا وسهل الاستخدام لتطبيقات التطوير والاختبار.

للتأكد من أن الكود يعمل بشكل صحيح، نَقِّد الأمر التالية في (Terminal):

```
python database.py
```

سيظهر ملف ال `test.db`:



إعداد قاعدة بيانات PostgreSQL :

في بيئات الإنتاج (Production Environments) أو في التطبيقات ذات الزيارات العالية (High-Traffic Applications)، غالبًا ما يُستخدم PostgreSQL نظرًا لقوّته واستقراره في معالجة البيانات بشكل متزامن وفعّال.

```
from sqlalchemy import create_engine

# PostgreSQL بقاعدة بيانات
SQLALCHEMY_DATABASE_URL = "postgresql://user:password@localhost/mydatabase"

# SQLAlchemy (Database Engine) إنشاء محرك
engine = create_engine(SQLALCHEMY_DATABASE_URL)
```

في هذه الحالة:

user: اسم المستخدم الخاص بقاعدة بيانات PostgreSQL الذي سيتم استخدامه للاتصال.

password: كلمة المرور المرتبطة بالمستخدم المذكور أعلاه.

localhost: عنوان الخادم (Host) الذي تعمل عليه قاعدة بيانات PostgreSQL (في حال كانت القاعدة على نفس الجهاز، يبقى localhost، أما إن كانت على خادم بعيد، فيُستبدل بعنوان IP أو اسم النطاق).

mydatabase: اسم قاعدة البيانات التي سيتم الاتصال بها داخل خادم PostgreSQL.

بهذا الشكل تُحدّد جميع تفاصيل الوصول إلى قاعدة البيانات لتتمكن SQLAlchemy من إدارة الاتصال وتنفيذ العمليات عليها بنجاح.

تعريف نماذج قاعدة البيانات والجداول (Defining Database Models and Tables)

تتيح لك SQLAlchemy تعريف نماذج قاعدة البيانات ك (class) في Python، بحيث تمثل هذه (class) الجداول المقابلة في قاعدة البيانات وتتولى إدارة عملية التحويل التلقائي بين كائنات بايثون والصفوف المخزنة في الجداول، مما يمكنك من التعامل مع البيانات بطريقة كائنية منظمة دون الحاجة إلى كتابة أوامر SQL مباشرة.

إنشاء نموذج (Defining a Model Class):

لإنشاء نموذج، نحتاج إلى تعريف صنف (Class) يرث من Base (وهي الفئة الأساسية التي توفرها مكتبة SQLAlchemy)، ثم نستخدم أنواع الأعمدة (Column Types) التي توفرها SQLAlchemy لتحديد الحقول داخل الجدول. المثال التالي يوضّح كيفية تعريف نموذج User يمثل جدول المستخدمين في قاعدة البيانات.

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

# إنشاء الفئة الأساسية لجميع النماذج
Base = declarative_base()

# تعريف نموذج المستخدم (User Model)
class User(Base):
    __tablename__ = "users" # اسم الجدول داخل قاعدة البيانات

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
```

يمثل User نموذجًا (Model) مرتبطًا بالجدول users في قاعدة البيانات، هذا النموذج يحدد بنية الجدول من خلال الحقول (Columns) التالية:

id: رقم تعريف فريد لكل مستخدم، يُستخدم كمفتاح أساسي (Primary Key) للتمييز بين الصفوف.

username و email: حقول نصية (String) تحتوي على اسم المستخدم وبريده الإلكتروني، مع وضع قيود تمنع التكرار (unique=True) لضمان عدم وجود مستخدمين بنفس الاسم أو البريد.

يتم تعريف كل حقل باستخدام دالة Column() من مكتبة SQLAlchemy لتحديد نوع البيانات (Integer, String, Boolean, ...). والخصائص المرتبطة بها مثل primary_key و unique و nullable و index.

أما index=True، فهي خاصية تُستخدم لإنشاء فهرس (Index) على الأعمدة المحددة، مما يجعل عمليات البحث والاستعلام عن البيانات أسرع وأكثر كفاءة.

وباستخدام declarative_base() يتم إنشاء الفئة الأساسية Base، والتي ترث منها جميع النماذج في المشروع، هذا يتيح لـ SQLAlchemy معرفة كيفية ربط الأصناف (Classes) بالجدول الفعلية في قاعدة البيانات، وبالتالي تحويل كائنات Python إلى صفوف داخل قاعدة البيانات والعكس بسهولة.

إنشاء الجداول قاعدة البيانات او التحويل الى قاعدة البيانات (Migrations) :

بمجرد أن تنتهي من تعريف النماذج التي شكلت بيئة قواعد البيانات الخاصة بك، يمكنك إنشاء الجداول المقابلة لها داخل قاعدة البيانات باستخدام الدالة Base.metadata.create_all() من مكتبة SQLAlchemy. حيث تقوم هذه الدالة بقراءة جميع النماذج المعرفة التي ترث من Base وإنشاء الجداول المطابقة لها في قاعدة البيانات إن لم تكن موجودة مسبقًا، وفي هذه الحالة تكون وكأنك أنشأت قواعد بيانات كما تفعل يدويًا.

لنرى كيف نرحل الهيكلية الى قواعد البيانات:

```
from sqlalchemy import create_engine
from models import Base

# مثال على الاتصال بقاعدة بيانات SQLite
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

# إنشاء محرك قاعدة البيانات
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # مطلوب فقط مع SQLite
)

# إنشاء جميع الجداول المعروفة في النماذج داخل قاعدة البيانات
Base.metadata.create_all(bind=engine)
```

تقوم الدالة `create_all()` بفحص الجداول في قاعدة البيانات، فإذا لم تكن موجودة تقوم بإنشائها تلقائيًا، حيث تقوم بمسح جميع الأصناف (Classes) مثل `User` وغيرها التي ترث من `Base`، ثم تنشئ الجداول المقابلة لها داخل قاعدة البيانات `test.db` في حال عدم وجودها مسبقًا.

إنشاء واجهة CRUD لعمليات قاعدة البيانات

الآن بعد أن قمنا بإعداد الاتصال بقاعدة البيانات (Database Connection) وتعريف النماذج (Mod-) (`els`)، يمكننا إنشاء واجهة برمجة تطبيقات (API) تقوم بتنفيذ عمليات CRUD (إنشاء `Create`، قراءة `Read`، تحديث `Update`، حذف `Delete`) على قاعدة البيانات (Database).

إطار العمل `FastAPI` يجعل من السهل إنشاء المسارات (Routes) التي تتفاعل مع قاعدة البيانات (Da-) (tabase) وتنفذ عمليات مثل الإدراج (`Insert`)، التحديث (`Update`)، والحذف (`Delete`) على السجلات (Records).

إنشاء جلسة قاعدة البيانات (Creating the Database Session):

تستخدم مكتبة SQLAlchemy الجلسات (Sessions) للتفاعل مع قاعدة البيانات (Database)، تُدير جلسة (Session) العمليات الخاصة بواحد أو أكثر من الاستعلامات (Queries) على قاعدة البيانات.

يمكنك إنشاء جلسة (Session) باستخدام الكائن sessionmaker، والذي يوفّر نطاقاً (Scope) محدداً للعمليات التي تُنفَّذ على قاعدة البيانات (Database Operations).

لنرى مثلاً لنفهم أكثر:

```
from sqlalchemy.orm import sessionmaker
from sqlalchemy import create_engine

SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

# إنشاء محرك قاعدة البيانات
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False} # ضروري فقط مع SQLite
)

# إنشاء الجلسة لإدارة الاتصال بالقاعدة
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

نحن هنا نقوم بإنشاء ما يُسمّى محرك قاعدة البيانات (Database Engine) هذا المحرك هو المسؤول عن فتح الاتصال الفعلي بين تطبيقنا وقاعدة البيانات (Database Connection) حتى نتمكن من تنفيذ الأوامر مثل الإضافة (Insert)، التحديث (Update) أو الحذف (Delete).

المعامل connect_args={"check_same_thread": False} يُستخدم فقط في حالة قاعدة بيانات SQLite بشكل افتراضي، SQLite لا تسمح باستخدام نفس الاتصال في أكثر من "خيط" (Thread) داخل التطبيق، وهذا قد يسبب خطأ في تطبيقات الويب التي تستخدم أكثر من طلب في نفس الوقت، لذلك نضع هذا الخيار لجعل SQLite أكثر مرونة ونسمح باستخدام نفس الاتصال في أكثر من خيط.

في السطر الذي يليه، يُنشئ ما يُعرف بـ مصنع الجلسات (Session Factory) باسم SessionLocal ببساطة، هذا المصنع هو الطريقة التي نستخدمها لإنشاء جلسة اتصال جديدة (Session) في كل مرة نحتاج فيها إلى التعامل مع قاعدة البيانات.

داخل هذا السطر يوجد ثلاث إعدادات مهمة:

autocommit=False <

يعني أن التغييرات لا تُحفظ مباشرة في قاعدة البيانات. بدلاً من ذلك، نقوم نحن باستدعاء الدالة commit() يدويًا عندما نتأكد أن كل شيء صحيح، وهذا يُعطي أمانًا أكبر.

autoflush=False <

هذا يمنع إرسال التغييرات بشكل تلقائي قبل تنفيذ الاستعلامات (queries)، حتى لا يحدث تحديث غير متوقع في البيانات.

bind=engine <

هذا يخبر الجلسة أن تعتمد على نفس المحرك (Engine) الذي أنشأناه في السطر السابق للاتصال بقاعدة البيانات.

ببساطة واختصار:

engine = هو الخط الذي يربط تطبيقك بقاعدة البيانات. <

SessionLocal = هو الأداة التي تستخدمها لاحقًا لإنشاء جلسة مؤقتة تتعامل من خلالها مع البيانات، ثم تُغلق بعد الانتهاء من العمليات. <

التبعيات للحصول على جلسة قاعدة البيانات (Dependency to Get the Database Session):

في تطبيق FastAPI، من أفضل الممارسات إنشاء تبعيات (Dependencies) لجلسات قاعدة البيانات (Database Sessions) حتى يضمن الإطار أن كل طلب (Request) يحصل على جلسة مستقلة (Independent Session) خاصة به، تُنشأ عند بداية الطلب وتُغلق تلقائيًا بعد اكتماله هذه الطريقة تمنع تداخل الطلبات مع بعضها، وتحافظ على استقرار وأمان الاتصال بقاعدة البيانات (Database Connection) من خلال إدارة الجلسات بطريقة منظمة وآمنة.

مثال:

```
from fastapi import Depends
from sqlalchemy.orm import Session
from .database import SessionLocal # استيراد من ملف إعداد قاعدة البيانات

# التابع المسؤول عن إنشاء جلسة قاعدة البيانات وإغلاقها بعد الاستخدام
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

الكلمة المفتاحية **yield** تُستخدم هنا لتوفير الجلسة للمسار (Route) الذي يحتاجها أثناء تنفيذ الطلب، وبعد انتهاء العملية سواء نجحت أم فشلت يتم تنفيذ الجزء داخل **finally** الذي يستدعي **db.close()** لإغلاق الجلسة وتحرير الاتصال بقاعدة البيانات.

إنشاء عمليات CRUD :

يمكننا الآن تعريف الدوال الخاصة بعمليات CRUD، حيث تتفاعل هذه الدوال مع جلسة قاعدة البيانات (Database Session) لتنفيذ العمليات الأساسية مثل الإنشاء (Create)، القراءة (Read)، التحديث (Update)، والحذف (Delete) على السجلات (Records) داخل قاعدة البيانات (Database).

عملية الإنشاء: <

```
from sqlalchemy.orm import Session
from .models import User

def create_user(db: Session, username: str, email: str):
    # إنشاء كائن مستخدم جديد من نموذج User
    db_user = User(username=username, email=email)

    # إضافة الكائن إلى جلسة قاعدة البيانات
    db.add(db_user)

    # في قاعدة البيانات (commit) تنفيذ عملية الحفظ
    db.commit()

    # (الذي تم توليده تلقائيًا id تحديث الكائن ليعكس القيم النهائية من القاعدة مثل)
    db.refresh(db_user)

    # إعادة الكائن بعد الإنشاء
    return db_user
```

< عملية القراءة:

```
from sqlalchemy.orm import Session
from .models import User

def get_user_by_username(db: Session, username: str):
    # البحث في قاعدة البيانات عن مستخدم بالاسم المحدد
    return db.query(User).filter(User.username == username).first()
```

< عملية التحديث:

```
from sqlalchemy.orm import Session
from .models import User

def update_user_email(db: Session, user_id: int, new_email: str):
    # user_id البحث عن المستخدم بواسطة
    db_user = db.query(User).filter(User.id == user_id).first()

    # إذا وُجد المستخدم، يتم تحديث البريد الإلكتروني
    if db_user:
        db_user.email = new_email
        # حفظ التغييرات في قاعدة البيانات
        db.commit()
        # تحديث الكائن من القاعدة بعد التعديل (للحصول على البيانات المحدثة)
        db.refresh(db_user)

    # إعادة الكائن المعدّل
    return db_user
```

< عملية الحذف:

```
from sqlalchemy.orm import Session
from .models import User

def delete_user(db: Session, user_id: int):
    # البحث عن المستخدم بناءً على المعرف
    db_user = db.query(User).filter(User.id == user_id).first()

    # إذا وُجد المستخدم، يتم حذفه من قاعدة البيانات
    if db_user:
        db.delete(db_user)

        # تنفيذ عملية الحذف فعليًا في قاعدة البيانات
        db.commit()

    # (إعادة الكائن المحذوف) (قبل الحذف من الجلسة)
    return db_user
```

إنشاء نقاط نهاية (Creating FastAPI Endpoints):

الآن بعد أن أصبح لدينا دوال CRUD، يمكننا إتاحتها كمسارات (Routes) في إطار FastAPI بحيث يمكن الوصول إليها عبر طلبات HTTP لتنفيذ العمليات المختلفة على قاعدة البيانات.

```
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from .models import User
from .crud import create_user, get_user_by_username, update_user_email, delete_user
from .database import get_db
app = FastAPI()

# إنشاء مستخدم جديد (Create User)
@app.post("/users/")
def create_user_endpoint(username: str, email: str, db: Session = Depends(get_db)):
    # استدعاء دالة الإنشاء من ملف crud.py
    return create_user(db=db, username=username, email=email)

# قراءة مستخدم باستخدام الاسم (Read User)
@app.get("/users/{username}")
def read_user(username: str, db: Session = Depends(get_db)):
    # البحث عن المستخدم في قاعدة البيانات
    user = get_user_by_username(db=db, username=username)
    # إذا لم يوجد المستخدم، نرجع خطأ 404
    if not user:
        raise HTTPException(status_code=404, detail="المستخدم غير موجود")
    # في حال وجوده نرجع بياناته
    return user

# تحديث البريد الإلكتروني للمستخدم (Update User Email)
@app.put("/users/{user_id}")
def update_user_endpoint(user_id: int, new_email: str, db: Session = Depends(get_db)):
    # استدعاء دالة التحديث وتمرير البيانات الجديدة
    user = update_user_email(db=db, user_id=user_id, new_email=new_email)
    # إذا لم يوجد المستخدم نرجع خطأ 404
    if not user:
        raise HTTPException(status_code=404, detail="المستخدم غير موجود")
    # في حال نجاح العملية نرجع بيانات المستخدم بعد التحديث
    return user

# حذف مستخدم من قاعدة البيانات (Delete User)
@app.delete("/users/{user_id}")
def delete_user_endpoint(user_id: int, db: Session = Depends(get_db)):
    # استدعاء دالة الحذف من ملف crud.py
    user = delete_user(db=db, user_id=user_id)
    if not user:
        raise HTTPException(status_code=404, detail="المستخدم غير موجود")
    return {"detail": "تم حذف المستخدم بنجاح"}
```


في هذا المثال:

- المسار /users/ يُستخدم لإنشاء مستخدم جديد (Create a new user).
- المسار /users/{username}/ يُستخدم لجلب تفاصيل المستخدم من خلال اسم المستخدم (Retrieve user details by username).
- المسار /users/{user_id}/ PUT يُستخدم لتحديث البريد الإلكتروني للمستخدم (Update the user's email).
- المسار /users/{user_id}/ DELETE يُستخدم لحذف المستخدم (Delete the user).

في هذا القسم، تعلمنا كيفية ربط إطار FastAPI بقاعدة البيانات باستخدام SQLAlchemy، حيث قمنا بإعداد الاتصال بقاعدتي SQLite و PostgreSQL، وتعريف النماذج والجداول باستخدام SQLAlchemy ORM، وإنشاء واجهة API لتنفيذ عمليات الإنشاء (Create)، القراءة (Read)، التحديث (Update)، والحذف (Delete) على قاعدة البيانات.

من خلال دمج SQLAlchemy مع FastAPI، يصبح من السهل التعامل مع قواعد البيانات العلائقية بكفاءة وإدارة البيانات داخل التطبيق بمرونة عالية؛ هذه المعرفة تُعد أساساً مهماً لبناء واجهات برمجية تطبيقات (APIs) قوية وديناميكية قادرة على العمل بفاعلية مع البيانات العلائقية.

❖ التعامل مع الأخطاء أثناء عمليات قاعدة البيانات

يُعد التعامل مع الأخطاء أثناء عمليات قاعدة البيانات أمراً أساسياً للحفاظ على سلامة البيانات (Data Integrity) وتقديم ملاحظات واضحة ومفيدة للعميل (Meaningful Feedback to the Client) فخلال التفاعل مع قاعدة البيانات (Database Interaction)، قد تواجه أخطاء شائعة مثل تعارض البيانات (Data Conflicts)، أو إدخال قيم مكررة (Duplicate Entries)، أو فشل الاتصال بقاعدة البيانات، أو محاولة الوصول إلى سجل غير موجود.

هناك بعض أنواع الأخطاء الشائعة التي قد تواجهها أثناء التفاعل مع قاعدة البيانات:

◀ **Integrity Errors:** تحدث عند انتهاك قيود قاعدة البيانات (Database Constraints) مثل محاولة إدخال قيمة مكررة في حقل فريد (Unique Field) أو إدخال بيانات تخالف قيود المفتاح (Foreign Key Constraints).

مثال:

```
from sqlalchemy.exc import IntegrityError
from fastapi import HTTPException
from sqlalchemy.orm import Session
from .models import User

def create_user(db: Session, username: str, email: str):
    try:
        # إنشاء كائن مستخدم جديد
        db_user = User(username=username, email=email)
        # إضافة المستخدم الجديد إلى الجلسة
        db.add(db_user)
        # تخزين البيانات في قاعدة البيانات (Commit) تنفيذ عملية الحفظ
        db.commit()
        # تحديث id الكائن بعد الحفظ للحصول على البيانات النهائية (مثل)
        db.refresh(db_user)
        # إرجاع المستخدم الذي تم إنشاؤه
        return db_user
    except IntegrityError:
        # في حال وجود خطأ مثل تكرار البيانات (Rollback) التراجع عن العملية
        db.rollback()
        # رسالة مفهومة للمستخدم HTTP رفع استثناء
        raise HTTPException(
            status_code=400,
            detail="يوجد مستخدم بهذا الاسم أو البريد الإلكتروني بالفعل"
        )
```

أخطاء التشغيل (Operational Errors): تحدث هذه الأخطاء عند وجود مشكلات في الاتصال بقاعدة البيانات (Database Connection) مثل انتهاء مهلة الاتصال (Network Timeout) أو تعطل خادم قاعدة البيانات (Database Server Issues). يمكن عادةً التقاط هذه الأخطاء باستخدام معالجة الاستثناءات العامة (General Exception Handling).

مثال:

```
from sqlalchemy.exc import OperationalError
from fastapi import HTTPException
from sqlalchemy.orm import Session
from .models import User

def get_user_by_id(db: Session, user_id: int):
    try:
        # البحث عن المستخدم في قاعدة البيانات باستخدام المعرف
        user = db.query(User).filter(User.id == user_id).first()

        # إذا لم يتم العثور على المستخدم نرفع خطأ 404
        if not user:
            raise HTTPException(status_code=404, detail="المستخدم غير موجود")

        # في حال وجود المستخدم نُعيد بياناته
        return user

    except OperationalError:
        # في حال وجود مشكلة في الاتصال بقاعدة البيانات
        raise HTTPException(status_code=500, detail="خطأ في الاتصال بقاعدة البيانات")
```

التعامل مع أخطاء القيم (Handling Value Errors): تحدث هذه الأخطاء عندما تكون قيمة مطلوبة أو معلمة مفقودة (Missing Parameter) أو عند إدخال بيانات بتنسيق غير صالح (In-valid Data Format)، وفي هذه الحالة يمكن رفع استثناء من نوع ValueError للتنبيه إلى الخطأ في القيم المدخلة.

مثال:

```
from fastapi import HTTPException
from sqlalchemy.orm import Session
from .models import User

def update_user_email(db: Session, user_id: int, new_email: str):
    # التحقق من أن البريد الجديد غير فارغ
    if not new_email:
        raise ValueError("البريد الإلكتروني الجديد لا يمكن أن يكون فارغاً")

    # البحث عن المستخدم في قاعدة البيانات باستخدام المعرف
    user = db.query(User).filter(User.id == user_id).first()

    # إذا لم يتم العثور على المستخدم نرفع خطأ 404
    if not user:
        raise HTTPException(status_code=404, detail="المستخدم غير موجود")

    # تحديث البريد الإلكتروني للمستخدم
    user.email = new_email

    try:
        # تنفيذ التعديل في قاعدة البيانات
        db.commit()
    except Exception as e:
        # التراجع عن العملية في حال حدوث خطأ أثناء الحفظ
        db.rollback()
        raise HTTPException(status_code=500, detail=f"حدث خطأ أثناء تحديث البريد الإلكتروني {str(e)}")

    # تحديث الكائن بعد التعديل (اختياري)
    db.refresh(user)

    # إعادة المستخدم بعد التحديث
    return user
```

أفضل الممارسات في التعامل مع الأخطاء (Best Practices for Error Handling)

التراجع عن العمليات (Rollback Transactions):

يجب دائماً تنفيذ عملية التراجع (Rollback) عند حدوث أي خطأ أثناء التعامل مع قاعدة البيانات، لضمان عدم حفظ بيانات غير مكتملة أو غير صحيحة في قاعدة البيانات (Database).

استخدام معالجة استثناءات محددة (Use Specific Exception Handling):

من الأفضل التقاط الاستثناءات المحددة مثل `OperationalError` و `IntegrityError` للتعامل مع الأخطاء المعروفة بطريقة صحيحة، وهذا يتيح لك تقديم رسائل خطأ أكثر وضوحاً وفائدة للمستخدم (User-Friendly Error Messages).

تسجيل الأخطاء (Logging Errors):

من الضروري تسجيل الأخطاء التي تحدث أثناء عمليات قاعدة البيانات لأغراض التصحيح (Debugging) والمتابعة التشغيلية (Operational Monitoring)، مما يساعدك على تتبع المشكلات في بيئة الإنتاج واتخاذ الإجراءات المناسبة.

استجابات خطأ موحدة (Consistent Error Responses):

استخدم استثناء `HTTPException` في `FastAPI` لإرجاع استجابات خطأ موحدة مع رموز حالة مناسبة (Status Codes) مثل:

404 عند عدم العثور على مورد (Not Found)،

400 عند وجود خطأ في الطلب (Bad Request)،

500 عند حدوث خطأ داخلي في الخادم (Internal Server Error).

في هذه الوحدة "التعامل مع قواعد البيانات"، تعرّفنا على كيفية ربط تطبيق FastAPI بقاعدة البيانات باستخدام SQLAlchemy، وإدارة الجلسات والمعاملات (Sessions & Transactions) بكفاءة لضمان سلامة البيانات (Data Integrity) واستقرار النظام، كما تناولنا كيفية تنفيذ العمليات الأساسية (CRUD) والتعامل مع الأخطاء بطريقة احترافية تضمن موثوقية التطبيق وسلاسة أدائه.



الوحدة 6

أفضل الممارسات لتصميم واجهات البرمجة

أفضل الممارسات لتصميم واجهات البرمجة

❖ هيكل مشروع (FastAPI) قابل للتوسع

عند بناء واجهات برمجة حديثة (APIs) باستخدام إطار العمل (FastAPI)، من الضروري تنظيم المشروع بطريقة تضمن سهولة الصيانة (maintainability)، وقابلية التوسع (scalability)، وسهولة الفهم.

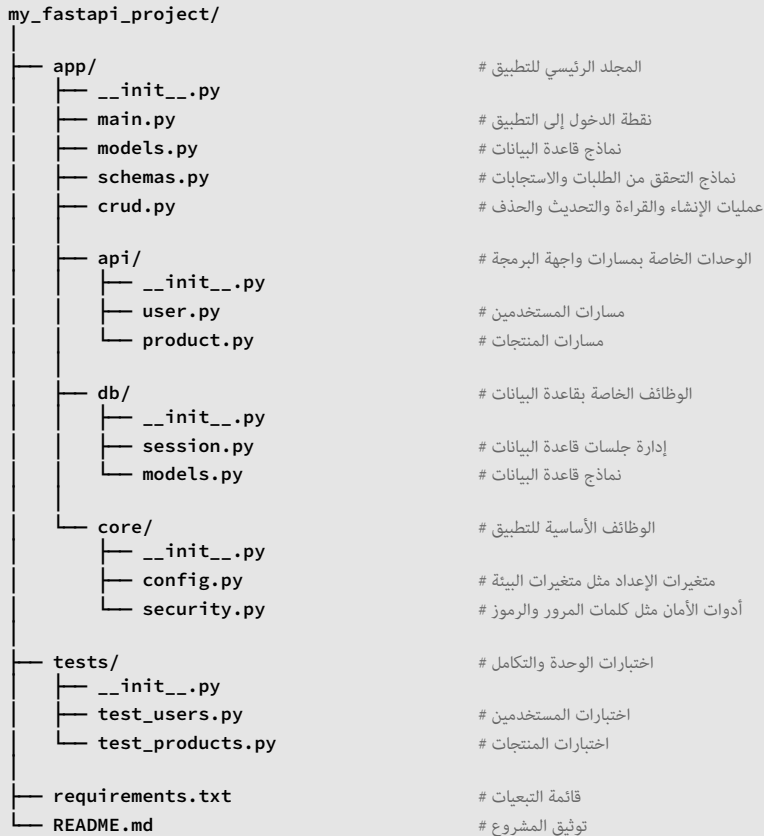
يسمح المشروع ذو الهيكل الجيد بتعاون أفضل بين أعضاء الفريق، ويسهل عمليات الاختبار (testing)، ويجعل النشر (deployment) أكثر سلاسة.

في هذا القسم، سنناقش كيفية تنظيم مشروع (FastAPI) إلى وحدات (modules)، واتباع أفضل الممارسات (best practices) لكتابة شيفرة نظيفة (clean) وسهلة الصيانة (maintainable code).

تنظيم المشروع إلى وحدات (Modules)

كلما نما تطبيقك المبنى باستخدام (FastAPI)، أصبح من المهم أكثر أن تُنظّمه بطريقة وحدوية (modular)، يضمن هذا الأسلوب القائم على الوحدات أن يكون كل جزء من التطبيق مستقلاً بذاته (self-contained) وسهل إعادة الاستخدام (reusable)، بحيث إن الهيكل المنظم والنظيف للمشروع يقلل من التعقيد (complexity) ويساعدك في الحفاظ على جودة عالية للكود (code quality)، مما يجعل التطبيق أسهل في التوسعة (extend) وتصحيح الأخطاء (debug).

فيما يلي مثال على كيفية تنظيم مشروع (FastAPI) بسيط إلى وحدات (modules):



الوحدات الأساسية (Key Modules) وأدوارها:

:main.py

هذا هو نقطة الدخول إلى تطبيقك المبني بإطار (FastAPI)، حيث يتم إنشاء كائن التطبيق (app in-) (stance) وإضافة جميع المسارات (routes) من هنا يتم تهيئة واجهة البرمجة (API) وتشغيل تطبيق (FastAPI).

:models.py

يحتوي على نماذج قاعدة البيانات (Database Models) سواء تم استخدام (SQLAlchemy) أو أي (ORM) آخر، تمثل هذه النماذج بنية الجداول (tables structure) داخل قاعدة البيانات.

:schemas.py

يعرف نماذج (Pydantic) التي تُستخدم للتحقق من الطلبات الواردة (incoming requests) والاستجابات الصادرة (outgoing responses) كما توفر هذه النماذج توثيقاً تلقائياً (automatic documentation) عبر واجهتي (Swagger UI) و (ReDoc).

:crud.py

يحتوي على الدوال الخاصة بعمليات الإنشاء (Create)، القراءة (Read)، التحديث (Update)، والحذف (Delete) في قاعدة البيانات، توجد هنا أغلب منطق الأعمال (business logic)، بحيث يتم فصل طبقة التعامل مع البيانات عن طبقة المسارات (routes layer).

/api

يُستخدم هذا المجلد لتنظيم نقاط نهاية الواجهة (API endpoints) حسب الموارد (resources) مثل المستخدمين أو المنتجات، يُفترض أن يمثل كل ملف مجموعة من المسارات المترابطة (related routes). على سبيل المثال:

- الملف user.py يحتوي على المسارات الخاصة بإنشاء المستخدمين أو عرضهم أو حذفهم.
- الملف product.py يحتوي على المسارات الخاصة بالمنتجات.

/db

مسؤول عن إدارة الوظائف المتعلقة بقاعدة البيانات، وعن الملف session.py يدير إنشاء ومعالجة جلسات قاعدة البيانات (database sessions)، وعن الملف models.py داخل هذا المجلد يحتوي على نماذج قاعدة البيانات (database models) والمنطق الخاص بـ (ORM).

/core

يتضمن الإعدادات الأساسية (core configurations)، والأدوات العامة (utilities)، والمنطق الخاص بالأمان والمصادقة (security & authentication)، الملف config.py يحتوي على قيم الإعداد (config-ration values) مثل رابط قاعدة البيانات، مفاتيح السر (secret keys)، ومتغيرات البيئة (environment variables)، الملف security.py يحتوي على وظائف مثل توليد رموز (JWT) وتجزئة كلمات المرور (password hashing).

/tests

يحتوي على اختبارات الوحدة (unit tests) واختبارات (integration tests) الخاصة بتطبيقك المبنى بـ-) FastAPI يجب أن يقابل كل ملف اختبار جانباً معيناً من واجهة البرمجة، مثلاً: test_users.py لاختبار مسارات المستخدمين و test_products.py لاختبار مسارات المنتجات. تساعد هذه الاختبارات في ضمان عمل التطبيق كما هو متوقع واكتشاف الأخطاء مبكراً.

:requirements.txt

يحتوي على قائمة التبعيات (dependencies) الخاصة بالمشروع، مما يسهل إعداد بيئة التطوير أو الإنتاج بسرعة على أي جهاز آخر.

:README.md

ملف يحتوي على معلومات أساسية عن المشروع، مثل طريقة الإعداد (setup instructions)، أمثلة الاستخدام (examples)، وتوثيق واجهة البرمجة (API documentation).

كتابة كود نظيف وقابل للصيانة

إضافةً إلى تنظيم مشروعك في وحدات منطقية (logical modules)، فإن كتابة كود نظيف وقابلة للصيانة (clean, maintainable code) أمرٌ أساسي لنجاح المشروع على المدى الطويل، الاكواد النظيفة تتبع أنماطاً متسقة (consistent patterns)، ويسهل قراءتها وفهمها وتعديلها.

فيما يلي بعض أفضل الممارسات (best practices) التي يجب وضعها في الاعتبار:

➤ اتّبع مبدأ عدم التكرار (DRY Principle – Don't Repeat Yourself):

إذا وجدت نفسك تكتب نفس الكود أكثر من مرة، فأعد هيكلته ضمن دالة (function) أو فئة (class) قابلة لإعادة الاستخدام. على سبيل المثال، إذا كنت تكتب عدة مسارات (routes) تتفاعل مع قاعدة البيانات بطريقة متشابهة، فقم بعزل المنطق المشترك (abstract the logic) ضمن دوال قابلة لإعادة الاستخدام داخل ملف crud.py.

مثال:

```
# بدلاً من تكرار هذا المنطق في عدة مسارات #
user = db.query(User).filter(User.username == username).first()

# أنشئ دالة قابلة لإعادة الاستخدام:
def get_user_by_username(db: Session, username: str):
    return db.query(User).filter(User.username == username).first()

# ثم استخدمها عبر مساراتك:
user = get_user_by_username(db, username)
```

استخدم أسماء ذات معنى (Use Meaningful Names):

يجب أن تعبر الأسماء بوضوح عما تفعله الدالة (function) أو المتغير (variable) أو الفئة (class) على سبيل المثال، اسم دالة مثل `get_user_by_username` يوضح بجلاء أن الدالة تقوم بجلب مستخدم بناءً على اسم المستخدم، تجنّب الأسماء الغامضة مثل `process_data()` أو `handle()` لأنها تجعل الكود أصعب في الفهم والصيانة.

مثال:

```
# مثال سيئ
def process_data():
    ...

# مثال جيد
def get_user_by_username(db: Session, username: str):
    ...
```

تجنّب وضع المنطق البرمجي داخل ال (Views/Routes)

يُنصَح أن تحتوي دوال المسارات (route functions) في FastAPI (داخل ملف `main.py`) على أقل قدر ممكن من المنطق، ينبغي أن تستدعي هذه الدوال دوالاً قابلة لإعادة الاستخدام من ملفات مثل `crud.py` أو `models.py`، حافظ على نظافة مسارات واجهة البرمجة (API routes) من خلال نقل المنطق المعقّد إلى دوال منفصلة (helper functions) يمكن اختبارها بشكل مستقل.

على سبيل المثال، بدلاً من كتابة منطق معقد داخل معالج المسار (route handler)، يمكنك استدعاء دالة مساعدة كما يلي:

```
# بدلاً من وضع المنطق هنا داخل المسار
@app.post("/users/")
def create_user(username: str, email: str, db: Session = Depends(get_db)):
    db_user = User(username=username, email=email)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

# الأفضل فصل منطق المعالجة في ملف crud.py
def create_user(db: Session, username: str, email: str):
    db_user = User(username=username, email=email)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
```

◀ فصل الإعدادات عن الكود:

قم بتخزين جميع المعلومات الحساسة والمخصصة لبيئة التشغيل مثل عناوين قواعد البيانات (da- base URLs)، ومفاتيح السرّ (secret keys)، والإعدادات الأخرى (configurations) في ملفات منفصلة مثل config.py أو في متغيرات البيئة (environment variables)، تجنب إدخال هذه القيم مباشرة داخل الكود (hardcoding) في مشروعك، لأن ذلك يجعل التطبيق أقل أماناً ويصعب عملية نقله بين بيئات مختلفة مثل التطوير والاختبار والإنتاج.

مثال (config.py):

```
import os
from dotenv import load_dotenv

# تحميل القيم من ملف .env
load_dotenv()

class Settings:
    # إعدادات قاعدة البيانات
    DATABASE_URL: str = os.getenv("DATABASE_URL")

    # مفتاح التشفير السري
    SECRET_KEY: str = os.getenv("SECRET_KEY")

    # وضع التشغيل (development, testing, production)
    ENVIRONMENT: str = os.getenv("ENVIRONMENT", "development")

    # إعدادات أخرى مثل صلاحية الرموز أو البريد الإلكتروني
    ACCESS_TOKEN_EXPIRE_MINUTES: int = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30))

    # إنشاء كائن إعدادات عام يمكن استدعاؤه في أي مكان داخل التطبيق
    settings = Settings()
```

يمكنك تحميل الإعدادات داخل ملف config.py ثم استيرادها عند الحاجة في بقية أجزاء المشروع، يسمح لك ذلك بفصل الإعدادات عن الكود وجعلها أسهل في التعديل والإدارة، خصوصًا عند العمل على بيئات مختلفة مثل التطوير (development) أو الإنتاج (production).

أكتب اختبارات الوحدة والتكامل (Write Unit and Integration Tests):

الاختبار عنصر أساسي للحفاظ على جودة عالية في قاعدة الكود (codebase)، اكتب اختبارات وحدة (unit tests) للتحقق من صحة عمل كل مكون على حدة، واختبارات تكامل (integration tests) للتأكد من أن التطبيق بأكمله يعمل كما هو متوقع.

استخدم إطار اختبار مثل pytest لأتمتة اختباراتك (automate your tests):

```
pip install pytest
```

يُفضل كتابة الاختبارات داخل مجلد tests/. فعلى سبيل المثال، الملف test_users.py سيكون مخصصًا لاختبار المسارات الخاصة بالمستخدمين (user-related API routes).

مثال على اختبار:

```
from fastapi.testclient import TestClient
from app.main import app

# إنشاء عميل اختبار من التطبيق الرئيسي
client = TestClient(app)

def test_read_main():
    # «/» إلى المسار الرئيسي GET إرسال طلب
    response = client.get("/")
    # (التحقق من أن حالة الاستجابة هي 200 (ناجح)
    assert response.status_code == 200
    # التحقق من أن البيانات المُعادَة صحيحة
    assert response.json() == {"message": "Hello, World!"}
```

وَتُقّى واجهة البرمجة (API) باستخدام (Swagger) و (ReDoc):

يُنشئ FastAPI تلقائيًا توثيقًا تفاعليًا لواجهة البرمجة (API) باستخدام Swagger UI و ReDoc. استخدم نماذج (Pydantic) المناسبة للتحقق من الطلبات (requests) والاستجابات (responses). بهدف إنشاء توثيق دقيق وواضح لواجهة البرمجة.

```
from pydantic import BaseModel
from fastapi import FastAPI

app = FastAPI()

class UserCreate(BaseModel):
    username: str
    email: str

@app.post("/users/")
def create_user(user: UserCreate):
    return {"username": user.username, "email": user.email}
```

سيقوم FastAPI تلقائيًا بإنشاء توثيق واجهة البرمجة (API documentation) باستخدام Swagger-UI و ReDoc.

ويمكن الوصول إليهما من خلال الروابط التالية:

user/docs/ تفتح واجهة Swagger UI التفاعلية، حيث يمكنك استعراض جميع المسارات (endpoints) الخاصة بالمستخدمين وتجربتها مباشرة.

user/redoc/ تفتح واجهة ReDoc، وهي واجهة توثيق منظمة ومفصلة تعرض جميع تفاصيل المسارات الخاصة بالمستخدمين بشكل واضح وسهل القراءة.

في هذا القسم، ناقشنا كيفية هيكلة مشروع FastAPI قابل للتوسُّع من خلال تنظيمه إلى وحدات (mod-ules) والحفاظ على كود نظيف وقابل لإعادة الاستخدام. تناولنا كيفية فصل المهام إلى ملفات مختلفة مثل `tests`، `models`، `schemas`، `crud`، `api`، `core` والممارسات في كتابة كود قابل للصيانة، مثل تجنُّب التكرار، واستخدام أسماء ذات معنى، والحفاظ على نظافة المسارات، وفصل الإعدادات، وكتابة الاختبارات لضمان موثوقية الكود، باتباع هذه الممارسات، يمكنك بناء واجهة برمجة (API) يسهل صيانتها وتوسيعها مع نمو التطبيق، حيث يضمن التنظيم الجيد والكود النظيف بقاء مشروع FastAPI فعَّالاً وسهل الإدارة وقادرًا على التكيف مع المتطلبات المتغيرة.

❖ الأمان في (FastAPI)

يَعَدُّ الأمان (Security) جانبًا أساسيًا في أي واجهة برمجة (API)، خصوصًا عند التعامل مع بيانات حساسة (sensitive data) أو عمليات تسجيل الدخول والمصادقة (authentication)، يوفر FastAPI مجموعة من الأدوات والاستراتيجيات التي تساعدك في حماية تطبيقك.

في هذا القسم، سنستعرض كيفية التعامل مع مشاركة الموارد عبر المصادر المختلفة (CORS - Cross-Origin Resource Sharing)، وتنفيذ المصادقة (authentication) باستخدام OAuth2، بالإضافة إلى كيفية حماية البيانات الحساسة (protect sensitive data) داخل تطبيقات FastAPI.

التعامل مع (CORS) وسياسات الأمان (Security Policies)

آلية Cross-Origin Resource Sharing هي نظام يسمح لمصفحات الويب بإرسال طلبات إلى نطاق مختلف عن النطاق الذي تم تحميل صفحة الويب منه، وهذا أمر شائع في تطبيقات JavaScript التي تعمل على المتصفح، حيث تحتاج إلى التواصل مع خادم (server) مستضاف على نطاق آخر.

بشكل افتراضي، يقوم FastAPI بحظر الطلبات القادمة من نطاقات مختلفة لأسباب أمنية، لكن في كثير من الحالات، ستحتاج إلى تفعيل CORS للسماح لتطبيقات الواجهة الأمامية (frontend applications) بالتفاعل مع الخادم الخلفي (backend) المبني باستخدام FastAPI.

تفعيل (CORS) في FastAPI:

يُسهّل FastAPI التعامل مع CORS من خلال استخدام CORSMiddleware المأخوذ من المكتبة starlette.middleware.cors. يمكنك إعداد هذا الـ Middleware لتحديد النطاقات (origins أو do-mains) المسموح لها بإرسال الطلبات إلى واجهة البرمجة الخاصة بك (API).

1. قم بتثبيت الحزم المطلوبة باستخدام الأمر التالي:

```
pip install "fastapi[all]"
```

2. أضف CORSMiddleware إلى تطبيقك في FastAPI كما يلي:

```
from fastapi import FastAPI
from starlette.middleware.cors import CORSMiddleware

# إنشاء تطبيق FastAPI
app = FastAPI()

# السماح بالوصول من نطاقات محددة (Origins)
origins = [
    "https://example.com", # نطاق محدد (الموقع الفعلي)
    "http://localhost", # بيئة التطوير المحلية
]

# إضافة CORS Middleware إلى التطبيق
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins, # قائمة النطاقات المسموح بها
    allow_credentials=True, # السماح بإرسال ملفات تعريف الارتباط (cookies)
    allow_methods=["GET", "POST", "PUT", "DELETE"], # أنواع الطلبات المسموح بها
    allow_headers=["*"], # السماح بجميع الرؤوس (headers)
)

# تعريف مسار بسيط لاختبار الاتصال
@app.get("/items/")
def read_items():
    return {"message": "Items retrieved successfully!"}
```

في المثال أعلاه:

- allow_origins يحدد النطاقات (domains) المسموح لها بإرسال الطلبات إلى واجهة البرمجة (API).
- allow_methods يحدد طرق HTTP المسموح بها من جانب العميل (مثل GET، POST، PUT، DELETE).
- allow_headers يضمن أن العميل يمكنه إرسال رؤوس الطلب (Headers) مخصصة (custom headers) عند الحاجة.

يساعد هذا الإعداد في الحفاظ على أمان واجهة البرمجة (API) مع السماح بالطلبات القادمة من النطاقات الموثوقة (trusted domains) فقط.

رؤوس الأمان الأخرى (Other Security Headers):

قد تحتاج أيضًا إلى ضبط رؤوس أمان إضافية (Security Headers) في تطبيقك المبني بـ FastAPI لضمان مستوى عالٍ من الحماية. يشمل ذلك إعداد رؤوس مثل (Content Security Policy — CSP)، و (Strict-Transport-Security — HSTS)، و (X-Frame-Options)؛ إذ تساهم هذه الرؤوس في التخفيف من هجمات مثل (Cross-Site Scripting — XSS) و (Clickjacking)، وبما أن FastAPI لا يوفر Middleware مدمجًا لكل هذه الرؤوس، يمكنك استخدام إطار Starlette أو مكتبات طرف ثالث لضبطها عند الحاجة.

المصادقة باستخدام (OAuth2)

تُعد المصادقة (Authentication) عملية التحقق من هوية المستخدم أو العميل (user or client)، يُعتبر OAuth2 معيارًا شائعًا لإعطاء صلاحيات (authorization)، إذ يتيح للتطبيقات الخارجية (third-party applications) الوصول إلى الموارد نيابةً عن المستخدم؛ يدعم FastAPI بروتوكول OAuth2 ويمكن دمجه بسهولة مع أدوات مثل JSON Web Tokens (JWT) لتنفيذ المصادقة القائمة على الرموز (token-based authentication).

تدقق كلمة المرور في OAuth2 مع رموز JWT

في هذا المثال سنوضح كيفية تنفيذ مصادقة OAuth2 باستخدام تدقق كلمة المرور (password flow)، حيث يقدم المستخدم بيانات الاعتماد (username, password)، وفي المقابل يتلقى رمز (JWT token) لاستخدامه في المصادقة المعتمدة على الرموز.

1. قم بتثبيت الحزم المطلوبة باستخدام الأمر التالي:

```
pip install "fastapi[all]" python-jose passlib
```

2. تعريف نماذج المصادقة:

```
from pydantic import BaseModel

class User(BaseModel):
    username: str
    password: str

class Token(BaseModel):
    access_token: str
    token_type: str
```

3. إنشاء دوال لمصادقة كلمة المرور:

```
from datetime import datetime, timedelta
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from passlib.context import CryptContext

import jwt  # حسب المكتبة المثبتة (from jose import jwt) أو PyJWT (import jwt) ملاحظة: يمكنك استخدام

# المسار الذي يوفر التوكن (OAuth2 استخراج رمز الوصول من رؤوس الطلب وفق)
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# سياق تجزئة كلمات المرور (Password Hashing Context)
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# القيم الأمنية (يفضل قراءتها من متغيرات البيئة وليس تثبيتها في الكود)
SECRET_KEY = "mysecretkey"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

قاعدة بيانات تجريبية (مستخدم واحد)
fake_users_db = {
    "johndoe": {
        "username": "johndoe",
        "hashed_password": pwd_context.hash("password123"),  # كلمة المرور بعد التجزئة
    }
}

التحقق من كلمة المرور: يقارن بين النص الصريح والتجزئة المخزنة
def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

# جلب مستخدم من قاعدة البيانات التجريبية
def get_user(db: dict, username: str):
    if username in db:
        return db[username]
    return None

مع وقت انتهاء صلاحية (JWT) إنشاء رمز وصول
def create_access_token(data: dict, expires_delta: timedelta = timedelta(minutes=15)) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + expires_delta
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

4. تنفيذ المسار /token لمصادقة المستخدمين

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from pydantic import BaseModel

# نفترض أن لديك الدوال التالية معرفة مسبقاً:
# - get_user(fake_users_db, username)
# - verify_password(plain_password, hashed_password)
# - create_access_token(data: dict)
# - fake_users_db, SECRET_KEY, ALGORITHM
# (إن لم تكن موجودة، أضفها كما في الأمثلة السابقة)

app = FastAPI()

# نموذج الاستجابة للتوكن
class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

@app.post("/token", response_model=Token)
async def login_for_access_token(
    form_data: OAuth2PasswordRequestForm = Depends()
):
    user = get_user(fake_users_db, form_data.username)
    # «password» وليس «hashed_password» انتبه: المفتاح الصحيح هو
    if not user or not verify_password(form_data.password, user["hashed_password"]):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # إنشاء رمز الوصول (JWT)
    access_token = create_access_token(data={"sub": form_data.username})
    return {"access_token": access_token, "token_type": "bearer"}
```

في الكود أعلاه:

يقوم المسار /token بعملية مصادقة المستخدم بناءً على اسم المستخدم (username) وكلمة المرور (password) المرسلين من خلال تدفق كلمة المرور في OAuth2 Password (OAuth2 Flow).

يتم إنشاء رمز وصول (JWT Access Token) يتضمّن وقت انتهاء صلاحية (expiration time) ويُعاد إلى العميل ضمن الاستجابة.

يُوفِّع هذا الرمز باستخدام مفتاح السرّ (SECRET_KEY) ويمكن استخدامه لاحقاً لمصادقة الطلبات التالية (subsequent requests) إلى واجهة البرمجة.

استخدام تدفّق كلمة المرور في داخل المسارات المحمية (Protected Endpoints):
لحماية المسارات باستخدام مصادقة OAuth2، يمكنك استخدام الدالة Depends مع الفئة OAuth2PasswordBearer لاستخراج رمز JWT من الطلب (request).

فيما يلي مثال على مسار محمي (protected route):

```
from fastapi import Depends, HTTPException, Security
from jose import jwt, JWTError # لمعالجة رموز jose مكتبة
from fastapi.security import OAuth2PasswordBearer

# لاستخراج التوكن من رؤس الطلب OAuth2 إعداد
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

SECRET_KEY = "mysecretkey"
ALGORITHM = "HS256"

# دالة لاستخراج المستخدم الحالي من التوكن
def get_current_user(token: str = Depends(oauth2_scheme)):
    credentials_exception = HTTPException(
        status_code=401,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
        return username
    except JWTError:
        raise credentials_exception

# مسار محمي يتطلب وجود توكن صالح للوصول
@app.get("/protected")
async def protected_route(current_user: str = Depends(get_current_user)):
    return {"message": f"Hello, {current_user}. You have access to this route!"}
```

في هذا المثال:

المسار `protected_route` محمي بواسطة آلية المصادقة (authentication) باستخدام OAuth-2PasswordBearer، مما يعني أنه لا يمكن الوصول إليه إلا من خلال رمز وصول (Access Token) صالح.

تقوم الدالة `get_current_user` بفك تشفير رمز JWT والتحقق من صلاحيته وأصالته (-authentic ity) قبل السماح بالوصول إلى المسار.

حماية البيانات الحساسة:

عند التعامل مع بيانات حساسة (sensitive data)، من الضروري اتخاذ إجراءات تضمن سرّيتها (-confidentiality) وسلامتها (integrity) وتوافرها (availability)، يوفّر FastAPI عدة استراتيجيات لحماية البيانات الحساسة سواء أثناء نقلها (transmission) أو أثناء تخزينها (storage).

استخدام بروتوكول HTTPS :

أحرص دائماً على استخدام HTTPS بدلاً من HTTP لضمان اتصال آمن، يقوم HTTPS بتشفير البيانات أثناء نقلها (data encryption in transit)، مما يحميها من هجمات الوسيط (Man-in-the-Middle – MITM).

في بيئة الإنتاج (production)، قم بإعداد خادم الويب (web server) مثل Nginx أو Apache، أو خادم ASGI الخاص بـ FastAPI مثل Uvicorn، ليعمل عبر HTTPS باستخدام شهادة أمان (SSL/TLS certificate) لضمان حماية الاتصال بين العميل والخادم.

تجزئة كلمات المرور (Hashing Passwords) :

لا تقم أبداً بتخزين كلمات المرور بنصّها الصريح (plaintext)، يجب دائماً تجزئة كلمات المرور (hashing) باستخدام خوارزمية قوية مثل bcrypt يتكامل FastAPI بشكل ممتاز مع مكتبة Passlib، التي تدعم العديد من خوارزميات التجزئة.

تشفير البيانات الحساسة (Encrypt Sensitive Data)

لأمان إضافي، يمكنك تشفير (encrypt) البيانات الحساسة المخزنة في قواعد البيانات (databases) لضمان بقائها آمنة حتى إذا تم اختراق القاعدة (compromised)، استخدم مكتبات مثل (cryp-tography) لتنفيذ التشفير (encryption) وفك التشفير (decryption).

استخدم متغيرات البيئة (Environment Variables)

قم بتخزين بيانات الحساسة مثل مفاتيح السر (secret keys) وبيانات اتصال قاعدة البيانات (database credentials) في متغيرات البيئة (environment variables) بدلاً من تضمينها مباشرة في الكود (hardcoding)، استخدم مكتبات مثل python-dotenv لتحميل هذه المتغيرات من ملفات env. أثناء مرحلة التطوير، بحيث تبقى القيم الحساسة محمية ويسهل تغييرها بين بيئات التطوير والاختبار والإنتاج دون تعديل الكود.

في هذا القسم، استعرضنا أهم الممارسات الأمنية لتطبيقات FastAPI، حيث تناولنا كيفية التعامل مع CORS وسياسات الأمان عبر تفعيلها وضبطها للسماح بالطلبات بين النطاقات بشكل آمن، بالإضافة إلى إضافة رؤوس أمان (HTTP Security Headers) لحماية التطبيق، كما شرحنا تنفيذ المصادقة باستخدام OAuth2 عبر رموز JWT لحماية المسارات والتحقق من الرموز الصالحة بعد تسجيل الدخول بنجاح، وتطرقنا إلى حماية البيانات الحساسة باستخدام استراتيجيات مثل الاتصال الآمن عبر HTTPS، وتجزئة كلمات المرور (hashing)، وتشفير البيانات (encryption)، واستخدام متغيرات البيئة (environment variables) لتخزين المعلومات السرية، باتباع هذه الممارسات، يمكنك ضمان أن يكون تطبيقك المبنى بـ FastAPI آمناً وفعالاً في إدارة المصادقة وحماية البيانات الحساسة.



الوحدة | 7

بناء الخدمات المصرفية باستخدام FastAPI

بناء الخدمات المصغرة باستخدام FastAPI

❖ تصميم الخدمات المصغرة باستخدام FastAPI

تُعد هندسة الخدمات المصغرة (Microservices Architecture) أسلوبًا في تصميم البرمجيات يقوم على بناء التطبيق كمجموعة من الخدمات المستقلة (independently deployable) ذات الترابط الضعيف (loosely coupled)، يختلف هذا النهج عن الهندسة الأحادية (Monolithic Architecture) التي تكون فيها جميع مكونات التطبيق مترابطة بإحكام داخل نظام واحد.

توفّر الخدمات المصغرة قابلية أعلى للتوسّع (scalability) ومرونة (flexibility) وسهولة في الصيانة (maintainability)، خصوصًا عند تطوير الأنظمة الموزعة (large-scale distributed systems)، تُعد FastAPI خيارًا مثاليًا لتصميم وتنفيذ الخدمات المصغرة، وبفضل أدائها العالي (performance) وبساطتها (simplicity) وسهولة تكاملها مع الأدوات الأخرى (ease of integration).

في هذا القسم، سنستعرض المبادئ الأساسية لهندسة الخدمات المصغرة وكيفية توسيع تطبيقات FastAPI بكفاءة.

مبادئ هندسة الخدمات المصغرة :

يتطلّب تصميم تطبيق قائم على الخدمات المصغرة (Microservices) فهمًا عميقًا وتطبيقًا عمليًا لعدد من المبادئ الأساسية، فيما يلي نستعرض هذه المبادئ التي تشكّل الأساس في تصميم وتطوير ونشر الخدمات المصغرة، وتوجّه كيفية بناء أنظمة مرنة، قابلة للتوسّع، وسهلة الصيانة.

1. استقلالية الخدمة (Service Independence):

الركيزة الأساسية لهندسة الخدمات المصغرة (Microservices) هي الاستقلالية (Independence)، ينبغي أن تكون كل خدمة (microservice) مستقلة بذاتها، تمتلك قاعدة كود خاصة (codebase)، وقاعدة بيانات منفصلة (database)، وآلية نشر مستقلة (deployment pipeline)، يُتيح هذا الأسلوب للفرق العمل على تطوير (develop)، واختبار (test)، ونشر (deploy) كل خدمة بمعزل عن الخدمات الأخرى، مما يعزّز سرعة التطوير وقابلية الصيانة كما يُفضّل أن لا تعتمد الخدمات على موارد أو خدمات مشتركة (shared resources or services) إلا في الحالات الضرورية فقط، للحفاظ على استقلالية كل خدمة ومرونتها في التطوير والتوسّع.

وهذا يعني أن تكون هناك:

◀ ترابط الضعيف (Loose Coupling):

يجب أن تتواصل كل خدمة مع الخدمات الأخرى من خلال واجهات برمجة محددة وواضحة (well-defined APIs)، غالبًا باستخدام بروتوكولات REST أو أنظمة المراسلة (message protocols) مثل Kafka أو RabbitMQ. هذا يعزز مرونة النظام ويقلل من الاعتماد المباشر بين الخدمات.

◀ ملكية البيانات (Data Ownership):

ينبغي أن تمتلك كل خدمة قاعدة البيانات الخاصة بها (own data store). يمكن للخدمات تبادل البيانات عبر واجهاتها (API communication)، ولكن لا يجب أن تشارك قاعدة بيانات واحدة مباشرة (share a single database)، وذلك لتجنب التعقيد وضمان استقلالية كل خدمة.

تُعد FastAPI أداة مثالية لبناء مثل هذه الخدمات المستقلة، بفضل قدرتها على إنشاء واجهات خفيفة (lightweight) ومعزولة (isolated APIs)، حيث يمكن تطوير كل خدمة، ونشرها، وتوسيعها بشكل مستقل عن بقية الخدمات.

2. مبدأ المسؤولية الواحدة (SRP - Single Responsibility Principle):

تُبنى الخدمات المصغرة (Microservices) حول وظائف أعمال محددة (specific business functions) بحيث تمتلك كل خدمة مسؤولية واحدة فقط، من خلال تطبيق مبدأ المسؤولية الواحدة (SRP)، تتركز كل خدمة على مهمة أو نطاق محدد (single task or domain)، مما يجعلها أسهل في الفهم وأسهل في الصيانة.

على سبيل المثال، قد تتولى إحدى الخدمات مهمة مصادقة المستخدمين (user authentication)، بينما تتولى أخرى معالجة المدفوعات (payment processing)، وأخرى تتعامل مع الإشعارات (notifications). الهدف من ذلك هو ضمان أن تمتلك كل خدمة غرضًا واحدًا محددًا بوضوح، مما يساهم في وضوح البنية وسهولة إدارة النظام وتوسّعه.

في FastAPI، يمكنك استخدام الموجهات (Routes) لتقسيم الكود بناءً على مجموعات منطقية (logical groupings)، مثل المستخدمين أو الطلبات أو المدفوعات، يساعد هذا التقسيم في جعل كل خدمة مصغرة (microservice) مقسمة بوضوح إلى أقسام مستقلة، مما يجعل الكود أنظف وأسهل في الصيانة.

3. قابلية التوسع (Scalability):

تُصمَّم الخدمات المصغَّرة (Microservices) بطبيعتها لتكون قابلة للتوسُّع (scalable)، وبما أن كل خدمة مستقلة عن الأخرى يمكن توسيعها أفقيًا (horizontal scaling) لتلبية زيادة الطلب، أي يمكن تشغيل عدَّة نسخ (instances) من نفس الخدمة دون التأثير على الخدمات الأخرى في النظام، مما يوفِّر مرونة وكفاءة عالية في التعامل مع تغيُّر حجم الاستخدام أو عدد المستخدمين.

في هندسة الخدمات المصغَّرة (Microservices Architecture)، يجب أن يركِّز التوسُّع (scal-) ing على الخدمات الفردية (individual services) بحسب مستوى الطلب (demand)، فعلى سبيل المثال، إذا كانت خدمة مصادقة المستخدمين (authentication service) تواجه ضغطًا عاليًا، بينما خدمة الإشعارات (notification service) لا تواجه ذلك، فيمكنك توسيع خدمة المصادقة فقط بشكل مستقل دون الحاجة إلى توسيع التطبيق بأكمله، يسمح هذا النهج باستخدام الموارد بكفاءة أكبر وتقليل التكلفة وتحسين أداء النظام ككل.

مع FastAPI، يمكنك الاستفادة من أدائه العالي (high performance) في معالجة عدد كبير من الطلبات بكفاءة، وعند تنفيذ التوسُّع الأفقي (horizontal scaling)، يمكنك توزيع الحمل (dis-tribute the load) بين عدَّة نسخ من تطبيق FastAPI، بل ويمكنك توزيعها على عدَّة خوادم (multiple servers) لضمان أداء مستقر وقدرة أكبر على تحمُّل الضغط.

4. التواصل عبر واجهات البرمجة (Communication via APIs):

في هندسة الخدمات المصغَّرة (Microservices Architecture)، تتواصل الخدمات مع بعضها البعض من خلال واجهات برمجة محددة وواضحة (well-defined APIs)، يحدث هذا التواصل عادةً عبر بروتوكول HTTP باستخدام واجهات RESTful APIs، أو من خلال أنظمة المراسلة (messaging systems) مثل Kafka أو RabbitMQ عند الحاجة إلى تواصل غير متزامن بين الخدمات.

يدعم FastAPI إنشاء توثيق تلقائي لواجهات البرمجة، مما يجعل تعريف واجهة كل خدمة (ser-vice API) واستهلاكها من قبل الخدمات الأخرى أمرًا سهلًا وواضحًا، كما يمكنك توفير تواصل آني (real-time communication) باستخدام WebSockets أو GraphQL لتمكين التفاعلات المباشرة بين الخدمات أو مع المستخدمين.

- **واجهات RESTful APIs:** يُسهِّل FastAPI إنشاء واجهات REST يمكن للخدمات المصغَّرة الأخرى استهلاكها بسهولة، مما يوفِّر تواصلًا منظمًا وواضحًا بين الخدمات.

- **التواصل غير المتزامن (Asynchronous Communication):** عند تصميم خدمات مصغرة تحتاج إلى تواصل غير متزامن، يتيح لك دعم async في FastAPI التعامل مع عدد كبير من الطلبات والرسائل بكفاءة عالية.

5. عزل الأعطال (Fault Isolation):

تضمن هندسة الخدمات المصغرة (Microservices Architecture) أنه في حال فشل إحدى الخدمات (service failure)، فإن ذلك لا يؤدي إلى تعطل النظام بأكمله، ويُعد عزل الأعطال (Fault Isolation) عنصرًا أساسيًا ومهمًا في تصميم الخدمات المصغرة، إذ يهدف إلى ضمان استمرارية عمل النظام حتى في حال حدوث أخطاء جزئية داخل خدمة واحدة فقط.

- **فقط قاطع الدارة (Circuit Breaker Pattern):** يمكنك تطبيق هذا النمط داخل خدماتك لاكتشاف حالات الفشل (detect failures) ومنع سلسلة الأعطال المتتالية (cascading failures) في النظام، فعند تكرار فشل خدمة معينة، يقوم قاطع الدارة بإيقاف المحاولات مؤقتًا حتى تستعيد الخدمة حالتها الطبيعية، مما يحافظ على استقرار النظام ككل.

- **إعادة المحاولة والمهل الزمنية (Retries & Timeouts):** عند استدعاء خدمات أخرى، تأكد من تطبيق معالجة أخطاء مناسبة (proper error handling) بالإضافة إلى آليات إعادة المحاولة (retries) وتحديد المهل الزمنية (timeouts) هذا يمنع الخدمات من الانتظار لفترات طويلة في حال تعطل خدمة أخرى، ويُسهّم في تحسين موثوقية النظام واستجابته.

يتكامل FastAPI بشكل ممتاز مع الوسائط الوسيطة (Middleware) التي تساعد على مراقبة الأعطال والتعامل معها بأسلوب سلس دون التأثير على باقي النظام، فعلى سبيل المثال، يمكنك استخدام مكتبات خارجية (third-party libraries) لتطبيق أنماط مثل قاطع الدارة (circuit breaking)، أو لتنفيذ إعادة المحاولة (retries) وتحديد المهل الزمنية (timeouts)، مما يعزّز من مرونة النظام (resilience) وقدرته على التعافي الذاتي عند حدوث الأخطاء.

6. أهمة عملية البناء والاختبار والنشر للتطبيق:

تستفيد الأنظمة المكونة من خدمات صغيرة (Microservices) من وجود خطوط آلية لعمليات البناء (Build) والاختبار (Test) والنشر (Deployment)، يجب أن تمتلك كل خدمة خطها الخاص لعمليات الدمج المستمر (Continuous Integration - CI) والنشر المستمر (Continuous Deployment - CD)، وهو ما يجعل عمليات البناء والاختبار والنشر تتم تلقائيًا دون تدخل يدوي.

هذا النهج يمكن الفرق (teams) من نشر نسخ جديدة من الخدمات بسرعة وأمان، دون التأثير على باقي أجزاء النظام، يمكن دمج خدمات FastAPI مع أدوات CI/CD مثل Jenkins و GitLab CI و GitHub Actions لجعل عملية النشر أكثر سلاسة وتنظيمًا.

7. إدارة البيانات الموزعة (Distributed Data Management)

هي طريقة لإدارة البيانات التي يتم تخزينها في أماكن أو خوادم (Servers) متعددة بدلاً من مكان واحد فقط، في النظام الأحادي (Monolithic System)، توجد عادة قاعدة بيانات واحدة (Sin-gle Database) تخدم جميع مكونات النظام، أما في الأنظمة المبنية على الخدمات الصغيرة (Microservices)، فالأمر مختلف؛ إذ تقوم كل خدمة بإدارة بياناتها الخاصة، وهذا يعني أن على كل خدمة أن تمتلك مخزن بيانات خاص بها (Data Store)، سواء كانت قاعدة بيانات من نوع SQL أو من نوع NoSQL، أو أي خيار تخزين آخر يناسب طبيعتها ووظيفتها، هذا الفصل في البيانات يجعل كل خدمة مستقلة تمامًا عن الأخرى، مما يزيد من المرونة (Flexibility) والقابلية للتوسع (Scalability) في النظام، لكنه في المقابل يتطلب تنسيقًا (Coordination) دقيقًا عند تبادل البيانات بين الخدمات المختلفة.

العمارة القائمة على الأحداث (Event-Driven Architecture):

لحفاظ على تزامن البيانات (Data Synchronization) بين الخدمات، يتم استخدام العمارة المعتمدة على الأحداث (Event-Driven Architecture) بشكل شائع، في هذا النوع من الأنظمة، تقوم الخدمات بإصدار أحداث (Emit Events) عندما يحدث تغيير معين في البيانات، وتقوم الخدمات الأخرى بالاستماع إلى هذه الأحداث والتفاعل معها عند الحاجة بهذه الطريقة، يتم تبادل المعلومات بشكل غير مباشر دون الحاجة لارتباط قوي بين الخدمات، مما يعزز المرونة (Flexibility) والاستقلالية (Decoupling).

تكرار البيانات (Data Duplication):

من الشائع وجود مستوى معين من تكرار البيانات (Data Duplication) في الأنظمة المبنية على الخدمات الصغيرة (Microservices) حيث يمكن لكل خدمة نسخ بعض عناصر البيانات (Replicate Data Elements) التي تحتاجها محلياً، وتقوم بمزامنتها (Sync) عند الضرورة، هذا الأسلوب يقلل من الاعتماد المباشر على الخدمات الأخرى، ويزيد من سرعة الأداء، حتى وإن أدى إلى وجود بعض البيانات المكررة عبر النظام.

توسيع تطبيقات (Scaling FastAPI Applications):

القابلية للتوسع (Scalability) هي واحدة من أهم مزايا الخدمات الصغيرة (Microservices)، وقد تم تصميم FastAPI للتعامل مع هذا الأمر بكفاءة عندما يحتاج تطبيقك إلى التوسع للتعامل مع زيادة في الحمل (Increased Load)، يمكنك تحقيق ذلك عن طريق زيادة عدد النسخ (Instances) من الخدمة، أو توزيع الخدمات عبر عدة توزيع الخدمات عبر عدة خوادم.

فيما يلي أفضل الممارسات لتوسيع (Scaling) تطبيقات FastAPI بشكل فعال:

1. التوسع الأفقي:

إحدى أبسط الطرق لتوسيع تطبيق FastAPI هي من خلال التوسع الأفقي (Horizontal Scaling)، حيث يتم نشر عدة نسخ (Instances) من نفس الخدمة، تُعد هذه الطريقة مفيدة بشكل خاص عندما تحتاج إلى التعامل مع حركة مرور عالية (High Traffic) أو عدد كبير من الطلبات (Requests).

موازنة التحميل (Load Balancing): قم بنشر عدة نسخ من تطبيق FastAPI خلف موازن تحميل (Load Balancer) مثل Nginx أو HAProxy، وذلك لتوزيع الطلبات الواردة (Incoming Requests) بشكل متساوٍ بين النسخ يساعد هذا على ضمان عدم إرهاق أي نسخة واحدة (Overwhelmed Instance) بالطلبات الزائدة، مما يحافظ على استقرار النظام وأدائه.

التحزيم بالحاويات باستخدام Docker: يمكن بسهولة تحزيم خدمات FastAPI داخل حاويات (Containers) باستخدام أداة Docker، مما يتيح لك تشغيل عدة نسخ (Instances) من الخدمة بسرعة وسهولة.

كوبرنيتس (Kubernetes): في الأنظمة الكبيرة والموزعة، تُعد كوبرنيتس (Kubernetes) منصة قوية لإدارة وتنظيم الخدمات الصغيرة (Microservices) من حيث النشر (De-ployment)، والتوسع (Scaling)، والإدارة (Management) عبر عناوين من الخوادم (Clusters of Servers)، يمكن نشر تطبيق FastAPI داخل بيئة Kubernetes لضمان القابلية للتوسع (Scalability) والمرونة العالية (Resilience).

2. المعالجة غير المتزامنة (Asynchronous Processing):

يُعد البرمجة غير المتزامنة (Asynchronous Programming) في FastAPI من أهم الأسباب التي تجعلها تعمل بكفاءة عالية تحت الضغط، يمكنك استخدام الدوال المعرفة بالكلمة المفتاحية `async def` لتنفيذ مهام مثل استعلام قواعد البيانات (Querying Databases) أو استدعاء واجهات برمجة التطبيقات الخارجية (Calling External APIs) دون حجب الخيط الرئيسي (Blocking the Main Thread) أثناء التنفيذ.

الإدخال والإخراج غير المتزامن: تدعم FastAPI بشكل مدمج الإدخال والإخراج غير المتزامن (Asynchronous I/O) من خلال مكتبة `asyncio` في Python، يُتيح ذلك للتطبيق التعامل مع عدد كبير من الطلبات (Requests) في الوقت نفسه دون أن يتعطل بسبب العمليات البطيئة مثل استعلامات قواعد البيانات (Database Queries) أو الطلبات الخارجية عبر HTTP (External HTTP Requests).

صفوف المهام (Task Queues): بالنسبة للمهام التي تستغرق وقتًا طويلاً، يمكنك استخدام صفوف المهام (Task Queues) مثل Celery أو RQ لتنفيذ المهام الخلفية (Background Tasks) بشكل غير متزامن (Asynchronously)، مما يساعد على إبقاء واجهة برمجة التطبيقات (API) سريعة الاستجابة.

3. التخزين المؤقت (Caching):

يمكن أن يُساهم التخزين المؤقت (Caching) بشكل كبير في تقليل الضغط (Reduce Load) على الخدمات وزيادة سرعة استجابتها (Responsiveness)، تدعم FastAPI عدة استراتيجيات للتخزين المؤقت مثل التخزين في الذاكرة (In-Memory Caching) باستخدام مكتبة `cache`-tools، أو استخدام أنظمة تخزين خارجية مثل Redis.

- ✦ تخزين استدعاءات واجهات البرمجة المؤقت (API Caching): قم بتخزين نتائج الاستدعاءات المتكررة أو المكلفة (Frequent or Expensive API Calls) مؤقتًا لتقليل الحاجة إلى إعادة الحسابات أو استعلامات قاعدة البيانات (Database Queries) المتكررة.
- ✦ تخزين قاعدة البيانات المؤقت (Database Caching): استخدم التخزين المؤقت (Caching) للاحتفاظ بنتائج الاستعلامات المتكررة (Frequently Accessed Queries)، مما يقلل الضغط على قاعدة البيانات ويحسن أداء النظام (Performance).

4. المراقبة والقياسات (Monitoring and Metrics):

للتوسع بفعالية تحتاج إلى مراقبة حالة الصحة والأداء لتطبيقاتك المبنية بـ FastAPI. تدعم Fa-stAPI أدوات مراقبة (Monitoring Tools) متعددة توفر لك رؤية واضحة (Visibility) حول أداء الخدمة (Service Performance)، واستخدام الموارد (Resource Usage)، ومعدلات الأخطاء (Error Rates).

- ✦ تُعد Prometheus و Grafana من الأدوات الشائعة لمراقبة الخدمات الصغيرة (Micro-services)، يمكنك عرض المقاييس (Expose Metrics) من تطبيق FastAPI باستخدام مكتبات مثل prometheus_fastapi_instrumentator.

- ✦ أما التتبع الموزع (Distributed Tracing) باستخدام أدوات مثل Jaeger، فيساعدك على تتبع تدفق الطلبات (Track the Flow of Requests) داخل النظام، وتحديد نقاط الاختناق (Bottlenecks)، وتحسين الأداء العام (Performance Optimization).

5. العمارة الموجهة نحو الخدمات الصغيرة (Microservice-Oriented Architecture):

عند تصميم تطبيقات FastAPI كخدمات صغيرة (Microservices)، احرص على تنظيمها بحيث تكون كل خدمة (Service) قابلة للنشر بشكل مستقل (Independently Deployable)، سهلة الصيانة (Maintainable)، وقابلة للتوسع (Scalable).

- ✦ بوابة واجهة البرمجة (API Gateway): في العمارة القائمة على الخدمات الصغيرة (Microservices Architecture)، يمكن لـ بوابة واجهة البرمجة (API Gateway) مثل Kong أو Nginx أو AWS API Gateway أن تعمل كوكيل عكسي (Reverse Proxy) يقوم بتوجيه الطلبات (Routing Requests) إلى الخدمات المناسبة داخل النظام.

◀ **اكتشاف الخدمات (Service Discovery):** عند التوسع، تحتاج الخدمات إلى معرفة الوجهة الصحيحة لإرسال الطلبات، تساعد أدوات مثل Consul أو Eureka في اكتشاف الخدمات (Service Discovery) وتحديد مواقعها بشكل ديناميكي داخل بيئة التشغيل.

كما ذكرنا يتطلب تصميم وتوسيع الخدمات الصغيرة باستخدام إطار FastAPI اتباع مجموعة من المبادئ الأساسية مثل استقلالية الخدمات، ووضوح الحدود، والقابلية للتوسع، والتواصل عبر واجهات البرمجة، يُعد FastAPI إطارًا مثاليًا لبناء خدمات صغيرة عالية الأداء، إذ يوفر دعمًا قويًا للبرمجة غير المتزامنة، وسهولة في النشر، والتكامل مع أدوات التوسيع والمراقبة، ومن خلال الالتزام بهذه المبادئ والاستفادة من قدرات FastAPI القوية، يمكن إنشاء خدمات صغيرة مرنة وقابلة للتوسع تلبي احتياجات التطبيقات الحديثة بكفاءة عالية.

❖ التواصل بين الخدمات (Inter-Service Communication):

في عمارة الخدمات الصغيرة (Microservices Architecture) تحتاج الخدمات المختلفة إلى التواصل (Communicate) مع بعضها لتنفيذ مهام معقدة، يمكن أن يحدث هذا التواصل بشكل متزامن (Syn-chronous) أو غير متزامن (Asynchronous) وفقًا لمتطلبات النظام (System Requirements)؛ قد تحتاج الخدمات إلى تبادل البيانات (Exchange Data) أو تحفيز إجراءات (Trigger Actions) في خدمات أخرى.

تستعرض هذه الفقرة طرقًا مختلفة لـ التواصل بين الخدمات (Inter-Service Communication) مع التركيز على نمطين شائعين: استخدام Redis و RabbitMQ للتواصل غير المتزامن (Asynchronous Communication)، والعمل مع بوابة واجهة برمجة التطبيقات (API Gateway) للتواصل المتزامن (Syn-chronous Communication).

استخدام Redis و RabbitMQ للتواصل بين الخدمات:

عند بناء الخدمات الصغيرة (Microservices)، من الضروري التفكير في كيفية تواصل الخدمات (Service Communication) مع بعضها، توجد نوعان رئيسيان من أنماط التواصل:

◀ التواصل المتزامن (Synchronous Communication) مثل واجهات REST APIs.

◀ التواصل غير المتزامن (Asynchronous Communication) مثل طوابير الرسائل (Message Queues).

في أنظمة الخدمات الصغيرة (Microservices)، غالبًا ما يُفَضَّل التواصل غير المتزامن (Asynchronous Communication) لأنه يحقق فك الارتباط بين الخدمات (Decoupling) ويحسن الأداء (Performance)، خصوصًا عندما تحتاج الخدمات إلى تنفيذ مهام في الخلفية (Background Tasks) أو التواصل بطريقة غير حابجة (Non-Blocking).

1. استخدام Redis للتواصل بين الخدمات:

يُعد Redis نظامًا مفتوح المصدر لتخزين البيانات في الذاكرة (In-Memory Data Store) وغالبًا ما يُستخدم كطبقة تخزين مؤقت (Caching Layer)، لكنه أيضًا يعمل كوسيط رسائل ممتاز (Message Broker) للتواصل بين الخدمات في عمارة الخدمات الصغيرة (Microservices Architectures)، يتيح Redis للخدمات التواصل بشكل غير متزامن (Asynchronously) وبكفاءة عالية،

مما يُساهم في تحسين سرعة النظام (System Performance) وتقليل زمن الاستجابة (Response Time).

- **نمط المراسلة بالنشر والاشتراك (Pub/Sub – Publish/Subscribe Messaging):** يدعم Redis غُط النشر والاشتراك (Pub/Sub) في المراسلة، حيث يمكن للخدمات نشر الرسائل (Publish Messages) في قنوات (Channels) معينة، بينما تقوم خدمات أخرى بالاشتراك (Subscribe) في هذه القنوات لتلقّي التحديثات، يُعد هذا النمط مناسبًا جدًا لـ الأنظمة القائمة على الأحداث (Event-Driven Architectures)، حيث تحتاج الخدمات إلى الاستجابة للأحداث (React to Events) التي تحدث داخل النظام بشكل فوري وفعال.
- **استخدام Redis مع FastAPI:** يمكن دمج Redis مع إطار FastAPI لتسهيل التواصل غير المتزامن (Asynchronous Communication) بين الخدمات، يمكنك استخدام مكتبة redis-py للتفاعل مع Redis من داخل تطبيق FastAPI، سواء لإرسال الرسائل أو استلامها بين الخدمات المختلفة.

فيما يلي مثال على استخدام Redis مع FastAPI في غُط النشر والاشتراك (Pub/Sub) للتواصل بين الخدمات:

```
import redis

import threading
from fastapi import FastAPI
app = FastAPI()

# Redis الاتصال بخادم
redis_client = redis.Redis(host='localhost', port=6379, db=0)

# نشر الرسائل في قناة معينة - الناشر (Publisher)
@app.post("/publish/{message}")
async def publish_message(message: str):
    redis_client.publish("my_channel", message)
    return {"status": "Message published"}

# الاشتراك في قناة واستقبال الرسائل - المشترك (Subscriber)
def redis_listener():
    pubsub = redis_client.pubsub()
    pubsub.subscribe("my_channel")
    for message in pubsub.listen():
        if message["type"] == "message":
            print(f"Received message: {message['data']}")

# تشغيل المستمع في خيط منفصل (يفضل أن يكون في الخلفية)
threading.Thread(target=redis_listener, daemon=True).start()
```

في هذا المثال:

- ◀ يقوم الناشر (Publisher) عبر المسار `publish/{message}/` بإرسال الرسائل (`Send Message`) إلى قناة في Redis تُسمى `my_channel`.
- ◀ بينما يقوم المشترك (Subscriber) بالاستماع (Listening) إلى هذه القناة ومعالجة الرسائل الواردة (Incoming Messages) بشكل غير متزامن (Asynchronously).

2. استخدام RabbitMQ للتواصل غير المتزامن:

يُعد RabbitMQ من أشهر وسطاء الرسائل (Message Brokers)، ويدعم أنماط طوابير الرسائل (Message Queuing) والنشر والاشتراك (Pub/Sub Messaging)، ويُستخدم RabbitMQ للتواصل غير المتزامن (Asynchronous Communication) بين الخدمات، خصوصًا عندما تحتاج الخدمات إلى وضع المهام في طوابير (Queue Tasks) لمعالجتها في الخلفية (Background Processing).

- ◀ نموذج طوابير الرسائل (Message Queuing): في هذا النموذج، يمكن للخدمات إرسال الرسائل (Send Messages) إلى طابور (Queue)، بينما تقوم الخدمات المستهلكة (Consumers) بمعالجة هذه الرسائل وفق وتيرتها الخاصة، يُوفّر RabbitMQ تسليمًا موثوقًا للرسائل (Reliable Message Delivery)، أي إن الرسائل لن تُفقد حتى في حال توقّف الخدمة المستقبلة مؤقتًا.

- ◀ استخدام RabbitMQ مع FastAPI: يمكن دمج FastAPI مع RabbitMQ باستخدام مكتبة pika، والتي توفّر واجهة سهلة للتعامل مع RabbitMQ من داخل تطبيق FastAPI.

فيما يلي مثال على إرسال رسالة إلى طابور (Queue) في RabbitMQ باستخدام FastAPI:

```
import pika
from fastapi import FastAPI

app = FastAPI()

# الاتصال بـ RabbitMQ
connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

# إنشاء الطابور (إن لم يكن موجودًا)
channel.queue_declare(queue='task_queue', durable=True)
```

```
# إرسال المهام إلى الطابور - المنتج (Producer)
@app.post("/send_task/{task}")
async def send_task(task: str):
    channel.basic_publish(
        exchange='', # التبادل الافتراضي
        routing_key='task_queue', # اسم الطابور
        body=task, # محتوى الرسالة
        properties=pika.BasicProperties(
            delivery_mode=2, # جعل الرسالة دائمة (Persistent)
        )
    )
    return {"status": "Task sent"}

connection.close()
```

في هذا المثال:

◀ يقوم المسار `send_task/{task}` بإرسال الرسائل (Messages) إلى طابور يسمى `task_queue`.

◀ يتم جعل الرسائل دائمة (Persistent) عبر الخاصية `delivery_mode=2`. أي سيتم حفظها على القرص (Saved to Disk) حتى تتم معالجتها من قبل المستهلك (Consumer).

لاستهلاك الرسائل، يتم إنشاء خدمة منفصلة (Separate Service) تستمع إلى طابور `task_queue` وتقوم بمعالجة الرسائل (Process Messages) بشكل غير متزامن (Asynchronously).

فيما يلي مثال على مستهلك (Consumer) يقوم بالاستماع إلى الرسائل:

```
import pika

# RabbitMQ الاتصال بـ
connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

# التأكد من وجود الطابور
channel.queue_declare(queue='task_queue', durable=True)

# تُنفَّذ عند وصول رسالة جديدة - دالة المعالجة (Callback)
def callback(ch, method, properties, body):
    print(f"Received task: {body.decode()}")
```

```
# معالجة الرسائل من الطابور - المستهلك (Consumer)
def start_consumer():
    channel.basic_consume(
        queue='task_queue',
        on_message_callback=callback,
        auto_ack=True # تأكيد تلقائي بعد استلام الرسالة
    )
    print('Waiting for messages. To exit press CTRL+C')
    channel.start_consuming()

# بدء تشغيل المستهلك
start_consumer()
```

في هذا المثال الخاص بالمستهلك (Consumer):

يقوم المستهلك بالاستماع إلى طابور task_queue ومعالجة الرسائل بشكل غير متزامن (-Asyn chronously) عند وصولها.

العمل مع بوابة واجهة البرمجة (Working with an API Gateway):

في بنية الخدمات المصغرة (Microservices Architecture)، تعمل بوابة واجهة برمجة التطبيقات (API Gateway) كنقطة دخول (entry point) للعملاء الخارجيين (external clients) وتقوم بتجميع الطلبات (aggregate requests) الموجهة إلى الخدمات المختلفة.

تتولى الـ (API Gateway) توجيه طلبات العملاء (client requests) إلى خدمات الخلفية المناسبة (backend services)، ويمكنها أيضاً توفير ميزات مثل المصادقة (authentication)، وتحديد المعدل (rate limiting)، وتسجيل الأحداث (logging)، وموازنة الأحمال (load balancing).

تساعد الـ (API Gateway) في مركزة الاتصال (centralize communication) وتوفّر نقطة دخول واحدة (single point of entry) إلى النظام، مما يجعل إدارة الخدمات المصغرة (microservices) وتوسيع نطاقها (scaling) أسهل وأكثر فعالية.

1. دور بوابة واجهة برمجة التطبيقات (Role of API Gateway):

تعمل بوابة واجهة برمجة التطبيقات (API Gateway) كوكيل عكسي (Reverse Proxy) للطلبات الواردة (incoming requests) في بنية النظام القائم على الخدمات المصغرة (-microservices architecture). فبدلاً من أن يتصل العملاء (clients) بكل خدمة منفصلة (individual service) بشكل مباشر، يتم توجيه جميع طلباتهم عبر الـ (API Gateway) التي تُعد الوسيط المركزي بين

العميل والخدمات المختلفة بعد استقبال الطلب، تقوم البوابة بإعادة توجيهه (forwarding the request) إلى الخدمة المصغرة المناسبة (relevant microservice)، ثم تجمع الاستجابة (response) وتعيدنها إلى العميل، يوفر هذا الأسلوب طبقة إضافية من التحكم (control) والأمان (security) والمرونة (flexibility)، إذ يُسهّل إدارة حركة البيانات (traffic management) ومراقبة الطلبات (request monitoring) وتطبيق سياسات مثل المصادقة (authentication) وتحديد المعدل (rate limiting) بطريقة موحّدة ومركزية.

- **التوجيه (Routing):** تتحمّل بوابة واجهة برمجة التطبيقات (API Gateway) مسؤولية توجيه الطلبات (routing requests) إلى الخدمة المصغرة المناسبة استناداً إلى عنوان الـ URL وطريقة الطلب (HTTP method) والرؤوس (headers).
- **المصادقة والتفويض (Authentication and Authorization):** يمكن لـ بوابة واجهة برمجة التطبيقات (API Gateway) التعامل مع الجوانب الأمنية مثل OAuth2 ورموز JWT tokens ومصادقة المستخدم، كما يمكنها تحرير بيانات المصادقة إلى الخدمات الخلفية (backend services) أو معالجتها بشكل مركزي (centrally) داخل النظام.
- **التجميع (Aggregation):** إذا كان العميل بحاجة إلى بيانات من عدة خدمات، يمكن لـ بوابة واجهة برمجة التطبيقات (API Gateway) تجميع الاستجابات (aggregate re-) sponses من عدة خدمات مصغرة (microservices) وإرسالها مرة أخرى على شكل استجابة واحدة (single response).

2. FastAPI و API Gateway:

على الرغم من أن FastAPI لا تحتوي على بوابة واجهة برمجة مدمجة (Built-in API Gateway)، إلا أنه يمكن استخدامها مع حلول بوابات واجهات البرمجة (API Gateway Solutions) مثل Kong أو Nginx أو AWS API Gateway لتوفير هذه الميزات.

تساعد هذه الحلول في توجيه الطلبات (Routing Requests) إلى الخدمات الصحيحة، بالإضافة إلى تقديم وظائف إضافية مثل الأمان (Security)، وتحديد المعدل (Rate Limiting)، والمراقبة (Monitoring)، وإدارة المصادقة (Authentication Management).

- **كونغ (Kong):** يُعد كونغ (Kong) بوابة واجهة برمجة التطبيقات (API Gateway) مفتوحة المصدر تُستخدم على نطاق واسع، حيث يمكنها توجيه الطلبات (route requests) إلى الخدمات المصغرة (FastAPI microservices)، كما توفر ميزات مثل المصادقة (authentication)، وموازنة الأحمال (load balancing)، والمزيد من الوظائف الأخرى.

إنجن إكس (Nginx): يُعتَبَر إنجن إكس (Nginx) خادماً عالي الأداء (high-performance web server) يمكنه أيضاً أن يعمل كواجهة برمجة التطبيقات (API Gateway)، حيث يتولى معالجة موازنة الأحمال (load balancing) والوكالة العكسية (reverse proxying) لخدمات FastAPI.

3. فوائد استخدام بوابة واجهة برمجة التطبيقات (API Gateway) في بنية الخدمات المصغرة (Microservices Architecture):

الإدارة المركزية (Centralized Management): تعمل بوابة واجهة برمجة التطبيقات (API Gateway) على مركزية التعامل مع الاهتمامات المشتركة (Cross-Cutting Concerns) مثل الأمان (Security)، وتحديد المعدّل (Rate Limiting)، والتخزين المؤقت (Caching)، والتسجيل (Logging)، مما يضمن الاتساق (Consistency) عبر جميع الخدمات.

الأمان (Security): يمكنها التعامل مع المصادقة (Authentication)، مثل OAuth2 أو التحقق من رموز JWT، قبل تمرير الطلبات إلى الخدمات الخلفية (Backend Services).

تحديد المعدّل والتحكم في التدفق (Rate Limiting and Throttling): يمكن لبوابات واجهة برمجة التطبيقات (API Gateways) فرض حدود على عدد الطلبات التي يمكن للمستخدم أو الخدمة إرسالها، وذلك لمنع التحميل الزائد على الخدمات الخلفية (Backend Services).

في النهاية، يُعَدُّ التواصل بين الخدمات (Inter-Service Communication) عنصراً محورياً في بنية الخدمات المصغرة (Microservices Architecture)، استخدام أدوات مثل Redis و RabbitMQ يمكن خدمات FastAPI من التواصل بشكل غير متزامن (Asynchronously)، مما يعزز الأداء وقابلية التوسع (Scalability)، ويوفّر Redis وسيلة سريعة باستخدام نمط Pub/Sub، بينما يضمن RabbitMQ معالجة موثوقة للمهام الخلفية عبر قوائم الانتظار (Message Queues)، أما API Gateway فتعمل كنقطة مركزية لتوجيه الطلبات وإدارة الجوانب المشتركة مثل الأمان (Security) وتحديد المعدّل (Rate Limiting). من خلال دمج هذه الأدوات والأنماط، يمكن بناء أنظمة فعّالة، قابلة للتوسع، وموثوقة باستخدام FastAPI.



المراجعة النهائية ومصادر التعلم الإضافية

المراجعة النهائية ومصادر التعلم الإضافية

مع اقترابنا من نهاية هذا الدليل لإتقان FastAPI باستخدام Python، دعنا نتوقف قليلاً للتأمل في المفاهيم والأدوات الأساسية التي تعلمتها عبر الفصول السابقة، بالإضافة إلى ذلك، سنقدم لك مصادر تعليمية إضافية تساعدك على تعميق فهمك لـ FastAPI ومواصلة رحلتك في تعلم وتطوير مهاراتك في عالم تطوير الويب (web development) وواجهات البرمجة (APIs) والأنظمة المصغرة (microservices).

❖ ملخص النقاط الرئيسية:

في هذا الكتاب، تناولنا مجموعة واسعة من الموضوعات المتعلقة بإطار FastAPI، بدءاً من أساسيات إعداد أول تطبيق، وصولاً إلى المواضيع المتقدمة مثل تكامل قواعد البيانات (database integration)، والعمليات غير المتزامنة (asynchronous operations)، وهندسة الأنظمة المصغرة (microservices architecture).

فيما يلي ملخص لأهم النقاط التي تمت مناقشتها في كل قسم:

1. لماذا FastAPI؟

يُعد FastAPI إطار عمل ويب عالي الأداء يدعم البرمجة غير المتزامنة، والتحقق التلقائي من البيانات، والتوثيق التفاعلي لواجهات البرمجة (interactive API documentation) عبر Swagger UI و ReDoc. إنه خيار ممتاز لبناء واجهات API حديثة تتطلب سرعة استجابة عالية (fast response times) وقدرة على التعامل مع عدد كبير من الطلبات المتزامنة (high concurrency).

2. البدء مع FastAPI

قمنا بتثبيت FastAPI و Uvicorn، وإعداد أول تطبيق، واستكشفنا كيفية تشغيله والتفاعل معه باستخدام Swagger UI و ReDoc لتوثيق الواجهة البرمجية، تُعد سهولة الإعداد والتوليد التلقائي للتوثيق من أبرز الميزات التي تميز FastAPI عن غيره من الأطر الأخرى.

3. بناء واجهات API باستخدام

تعلّمنا كيفية تعريف المسارات (routes) واستخدام طرائق HTTP المختلفة مثل GET، POST، PUT، DELETE، بالإضافة إلى التعامل مع المسارات الديناميكية (dynamic path parameters) ومعاملات الاستعلام (query parameters) إن بساطة تعريف المسارات في FastAPI تجعل تطوير واجهات برمجة نظيفة وسهلة الصيانة أمراً سريعاً وفعالاً.

4. التحقق من البيانات باستخدام Pydantic

استخدمنا نماذج Pydantic للتحقق من البيانات بشكل تلقائي، مما يضمن أن كل البيانات الواردة إلى واجهات API تكون منظّمة وصحيحة البنية (well-structured and validated) قبل تنفيذ أي منطق عمل، كما أن استخدام Pydantic يساعد في كشف الأخطاء والتعامل معها بسلاسة ويُبقي الكود أكثر أماناً وموثوقية.

5. تحسين الأداء باستخدام Async

استعرضنا الفرق بين العمليات المتزامنة (synchronous) وغير المتزامنة (asynchronous)، وكيف أن استخدام async def في FastAPI يحسّن الأداء بشكل كبير، خاصةً عند التعامل مع مهام الإدخال والإخراج الكثيفة (I/O-bound tasks)، إن دعم FastAPI للبرمجة غير المتزامنة يجعله قابلاً للتوسع وأكثر استجابة تحت الضغط.

6. العمل مع قواعد البيانات

غطينا تكامل FastAPI مع قواعد بيانات مثل SQLite و PostgreSQL باستخدام SQLAlchemy، my بشكل موسع، وتعلّمنا كيفية إنشاء واجهات (CRUD (Create, Read, Update, Delete)، وإدارة الجلسات (sessions) والمعاملات (transactions) بكفاءة، وهي مهارة أساسية لبناء تطبيقات ديناميكية تعتمد على البيانات (data-driven applications).

7. أفضل الممارسات في تصميم واجهات البرمجة

ناقشنا أفضل الممارسات في تصميم المشاريع باستخدام FastAPI، بما في ذلك تنظيم هيكل المشروع، وإدارة الجوانب الأمنية مثل المصادقة (authentication)، و CORS، وحماية البيانات الحساسة (sensitive data) كما تطرّقنا إلى مبادئ تصميم واجهات API النظيفة والأمنة والقابلة للتوسع (clean, secure, and scalable APIs) لضمان نجاح المشاريع على المدى الطويل.

8. بناء الأنظمة المصغرة باستخدام بناء الأنظمة المصغرة باستخدام FastAPI

تعلّمنا المبادئ الجوهرية لهندسة الأنظمة المصغرة (microservices architecture)، بما في ذلك كيفية تصميم هيكلية الخدمات المصغرة (structuring microservices)، وتوسيع نطاق تطبيقات FastAPI، وإدارة التواصل بين الخدمات (inter-service communication) باستخدام أدوات مثل Redis و RabbitMQ، إن اعتماد نهج الأنظمة المصغرة يساهم في إنشاء تطبيقات أكثر قابلية للصيانة (maintainable) وأعلى قابلية للتوسع (scalable) مع مرور الوقت.

❖ مصادر إضافية لإتقان:

لمواصلة التعلم وإتقان العمل باستخدام FastAPI، إليك مجموعة من المصادر الإضافية التي ستساعدك على توسيع معرفتك والبقاء على اطلاع بأحدث التطورات في بيئة FastAPI:

الموقع الرسمي لـ FastAPI :

يُعد الموقع الرسمي لإطار FastAPI المصدر الأشمل والأكثر تحديثاً لتعلم المزيد عن الإطار وفهمه بعمق، يوفر هذا التوثيق شروحات تفصيلية وأمثلة برمجية عملية، إلى جانب دروس معمّقة تغطي مواضيع متقدمة مثل:

❏ حقن التبعيات (Dependency Injection).

❏ المصادقة باستخدام OAuth2.

❏ دعم WebSockets.

وغيرها من الجوانب المتقدمة التي تساعد على بناء تطبيقات احترافية باستخدام FastAPI.

FastAPI على GitHub :

يُعتبر FastAPI على GitHub مصدرًا مهمًا للمطورين الراغبين في التعمّق في فهم الإطار من الداخل، يمكن من خلاله الاطلاع على الكود المصدري (source code)، والإبلاغ عن الأخطاء (bugs)، والمساهمة في تطوير الإطار، كما يتيح لك المستودع فرصة لتعلّم أفضل الممارسات البرمجية (best practices) وأنماط التصميم (design patterns) التي يعتمدها مطوّر FastAPI في بناء الإطار، من خلال دراسة الكود ومتابعة التحديثات، ستكتسب معرفة عملية بكيفية تصميم أطر عمل قابلة للتوسع وسريعة الأداء.

الدروس والدورات التعليمية:

تتوفر العديد من الدروس التعليمية والدورات عبر الإنترنت التي تغطي إطار FastAPI من المستوى المبتدئ إلى المتقدم، ومن أبرز المصادر الموصى بها التي تساعدك على تعلم الإطار خطوة بخطوة وبطريقة عملية:

❏ [FastAPI - The Complete Course 2025 \(Beginner + Advanced\)](#)

❏ [Learn Fast API With This ONE Project](#)

❖ استكشاف المواضيع المتقدمة:

مع ازدياد خبرتك في استخدام FastAPI، يُنصح بالانتقال إلى مواضيع أكثر تقدماً تساعدك على بناء تطبيقات أكثر قوة واحترافية، ومن أبرز هذه المواضيع:

1. WebSockets:

يُوفّر FastAPI دعماً قوياً لـ WebSockets، مما يمكنك من بناء تطبيقات تفاعلية لحظية (Real-Time Applications) مثل:

- ❏ تطبيقات الدردشة (Chat Applications).
- ❏ أنظمة البث المباشر للبيانات (Live Data Streaming).

تعلّم كيفية إنشاء واجهات API تفاعلية لحظية باستخدام WebSockets لتجعل تطبيقك أكثر استجابة وارتباطاً بالمستخدمين في الوقت الفعلي.

2. GraphQL:

يدعم FastAPI إنشاء واجهات GraphQL APIs كبديل حديث ومرن لواجهات REST التقليدية، استكشف كيفية دمج FastAPI مع مكتبات مثل **Graphene** لبناء واجهات GraphQL تتيح للمستخدمين استعلام البيانات بشكل ديناميكي وتحكم أكبر في البيانات المطلوبة.

3. تعلم الآلة مع FastAPI:

يمكن دمج FastAPI بسهولة مع نماذج تعلم الآلة (Machine Learning Models) لتقديم واجهات تنبؤية (Predictive APIs)، تعلّم كيفية:

- ❏ نشر نماذج تعلم الآلة (Deploy ML Models) عبر FastAPI.
- ❏ تقديم التنبؤات في الوقت الفعلي (Real-Time Predictions) من خلال واجهات API ذكية وسريعة.

يساعد هذا الدمج على تحويل نماذجك التحليلية إلى خدمات ذكية قابلة للاستخدام العملي في تطبيقات الويب أو الأنظمة الإنتاجية.

ها أنت الآن تصل إلى نهاية هذا الدليل، بعد رحلةٍ جمعت بين الفهم العميق والتطبيق العملي لإطار Fa-
 stAPI بلغة Python، لقد أصبحت تمتلك الأساس المتين لبناء واجهات برمجة قوية وسريعة (High-Per-
 formance APIs)، وقادراً على تصميم أنظمة مصغرة (Microservices) مرنة وقابلة للتوسع، لكن تذكر
 هذا ليس الختام، بل هو بداية طريقٍ جديد في رحلتك كمطورٍ يسعى نحو الإتقان، التعلم لا يتوقف،
 والممارسة المستمرة هي سرّ التميّز الحقيقي، ابن مشاريعك بيديك، واجه الأخطاء بشجاعة، وتعلّم من كل
 سطر كود تكتبه، شارك الآخرين علمك، وكن جزءاً من مجتمع يقدر الشغف والإبداع، فما بين كل تجربة
 وفكرة وكود جديد، تنمو مهارتك ويُصقل وعيك كمطورٍ يصنع التغيير امض بثقة، وواصل البناء، دمت بخير.



المراجع

توثيق FastAPI الرسمي (FastAPI Documentation):

يُعد التوثيق الرسمي لـ FastAPI من أكثر المصادر شمولاً وتحديثاً، يغطي كل ما تحتاج إليه من التثبيت الأولي وحتى الحالات المتقدمة (Advanced Use Cases)، ويُعتبر مرجعاً ممتازاً لفهم ميزات الإطار ووظائفه.

الرابط: [اضغط هنا](#)

توثيق Python الرسمي (Python Documentation)

يُعد التوثيق الرسمي للغة Python مرجعاً أساسياً لفهم أساسيات اللغة، وهو ضروري للعمل بفعالية مع FastAPI، يحتوي على شروحات تفصيلية لخصائص اللغة، بما في ذلك البرمجة غير المتزامنة (Asyn-chronous Programming)، وهي عنصر محوري في تطبيقات FastAPI.

الرابط: [اضغط هنا](#)

Pydantic Documentation

تُعد مكتبة Pydantic أحد أهم مكونات FastAPI، حيث تُستخدم في التحقق من البيانات (Validation) وتحويلها (Serialization)، يوضح التوثيق الرسمي كيفية عمل Pydantic واستخدامها بفعالية داخل تطبيقاتك.

الرابط: [اضغط هنا](#)

كتاب “Fluent Python” للمؤلف Luciano Ramalho

كتاب يُنصح به بشدة لكل من يرغب في إتقان Python بعمق، يتناول العديد من المواضيع المتقدمة، منها البرمجة غير المتزامنة، وهي مهارة أساسية لبناء واجهات API عالية الأداء باستخدام FastAPI.

كتاب “Python Microservices Development” للمؤلف Tarek Ziadé

يوفر هذا الكتاب أساساً قوياً لبناء الخدمات المصغرة (Microservices) باستخدام Python، مع تركيز خاص على أطر مثل Flask وFastAPI، يتضمن أمثلة عملية من الواقع تساعدك على بناء أنظمة قابلة للتوسع (Scalable Systems).

PostgreSQL Documentation

نظرًا لأن العديد من تطبيقات FastAPI تتفاعل مع قواعد البيانات — وخاصةً قواعد البيانات العلائقية مثل PostgreSQL — فإن هذا التوثيق يُعد مرجعًا مهمًا لفهم كيفية إدارة واستعلام البيانات بكفاءة.

الرابط: [اضغط هنا](#)

SQLAlchemy Documentation

تُستخدم مكتبة SQLAlchemy بشكل واسع مع FastAPI كأداة ORM (Object-Relational Mapping) للتعامل مع قواعد البيانات، يُعد التوثيق الرسمي ضروريًا لكل من يعمل على إدارة البيانات باستخدام SQLAlchemy ضمن تطبيقات FastAPI.

الرابط: [اضغط هنا](#)

كتاب “The Art of Scalability” للمؤلفين Michael T. Fisher و Martin L. Abbott

يتناول هذا الكتاب كيفية توسيع الأنظمة (Scalability) من حيث البنية المعمارية (Architecture) والفريق التقنية التي تقف خلف التطبيقات عالية الأداء.

Uvicorn Documentation

يُعد Uvicorn خادم ASGI الذي يستخدمه FastAPI لمعالجة طلبات HTTP. فهم كيفية عمل Uvicorn أمر ضروري لتحسين أداء تطبيقات FastAPI وضبطها للإنتاج.

الرابط: [اضغط هنا](#)

AsyncIO Documentation

تُعد مكتبة AsyncIO المدمجة في Python الأساس الذي تقوم عليه البرمجة غير المتزامنة في FastAPI. يشرح التوثيق كيفية كتابة كود غير متزامن بكفاءة، وهو عنصر أساسي في الأداء العالي الذي يتميز به الإطار.

الرابط: [اضغط هنا](#)